



论文查重/AIGC双擎报告

基本信息

检测版本：AIGC检测

论文标题：基于PVE的轻量级虚拟化实验平台设计

检测时间：2026-04-14 22:20:34

作者姓名：黄勇斌

时间范围：1990-01-01至2026-04-14

AIGC疑似率：**51.6%**

全文总字符数：**37,142**

您也可以查看逐句检测结果，片段的颜色表示该片段可能为AI生成的概率

- 黑色：低度疑似AI生成 AI生成的概率在0%~ 40%，判定为人类创作
- 橙色：中度疑似AI生成 AI生成的概率在40%~ 60%，疑似为AI生成
- 红色：高度疑似AI生成 AI生成的概率在60%~ 100%，判定为AI创作
- 灰色：不予检测片段 过短片段、标题及字符等不予检测的片段

本科毕业论文（设计）

题目 基于PVE的轻量级虚拟化实验平台设计
与实现

学院 人工智能学院

专业 网络工程

年级班级 22网络工程1班

学号 202213143002208

姓名 黄勇斌

指导老师 钟植任（职称:讲师）

目录

1.摘要 3

Abstract 4

绪论 6

1.1 研究背景与意义 6

1.2 国内外虚拟化管理平台的研究现状 7

1.3 主要研究内容与目标 8

2.系统相关技术介绍 10

2.1 前端核心技术	10
2.2 后端核心技术	12
2.3 PVE REST API	14
2.4 Paramiko 与 AsyncSSH	14
3.需求分析	15
3.1 系统可行性分析	15
3.2 功能需求分析	16
3.3 非功能性需求分析	20
4.系统设计	22
4.1 系统设计原则	22
4.2 系统总体设计	23
4.3 系统数据库设计	26
4.4 本章小结	46
5.系统实现	47
5.1 系统登录与权限管理模块的实现	47
5.2 仪表盘模块的实现	52
5.3 PVE 资源管理模块的实现	54
5.4 实验教学模块的实现	64
5.5 AI 助手模块的实现	68
6.系统测试	74
6.1 测试目的	74
6.2 测试环境	74
6.3 功能测试	75
6.4 本章小结	81
7.总结与展望	82
7.1 论文工作总结	82
7.2不足之处	83
7.3 未来展望	83
参考文献	85
致谢	87

1.摘要

伴随着虚拟化与云技术的广泛普及,各行业对 IT 资源的管控提出了更高要求。虽然开源平台 Proxmox VE 在私有云市场占据重要地位,但在面对多租户隔离、深度业务集成及用户 UI 设计时仍显乏力。为此,本文提出并实现了一个采用 Vue 3 框架与 Django 5 后端构建的云管平台。本研究不仅优化了 PVE 集群的操作门槛,还针对教学应用等实际场景进行了功能拓展,提升了系统的综合管理效能。

本系统遵循前后端分离的设计哲学。前端基于 Vue 3 组合式 API 与 Vite 构建引擎,依托 Vben Admin 框架打造了高性能的数据可视化管理界面;同时,集成 LogicFlow 绘制网络拓扑,并利用 xterm.js 与 noVNC 技术实现了沉浸式的远程终端交互。后端由 Django 5 配合 DRF 框架构建标准化的 RESTful 接口,底层存储采用 MySQL 数据库,并通过 PVE REST API 与 SSH 协议完成对物理资源的深度调度。此外,系统引入 ASGI 异步架构与 WebSocket 协议,确保了控制台数据传输的实时性与双向通信质量。本平台采用前后端分离的现代化架构进行开发。前端层面基于 Vue 3 的组合式 API、Vite 构建机制以及 Vben Admin 框架,构建了响应迅速、数据可视化的操作管理台;并深度集成了 LogicFlow 网络拓扑工具、xterm.js 和 noVNC 提供沉浸式的终端控制交互。后端层面采用 Django 5 与 DRF 构建 RESTful API,结合 MySQL 关系型数据库完成数据持久化,通过 PVE REST API 与 SSH 实现了基础设施的深度调度。同时,采用 ASGI 异步服务器与 WebSocket 协议,实现对实例控制台的实时对接与双向通信。

系统重点实现了四大核心功能:一是全生命周期的虚拟机与容器集群管理,支持基础监控与网络资源的拓扑可视化分配;二是

构建了基于 RBAC 模型的组织架构及多级细粒度权限控制体系,保障平台安全性并配备接口级操作审计流水;三是面向教育场景的实验教学双角色协同管理,为教师及学生提供在线实验环境的流式分发与 Markdown 实验报告编辑体验;四是创新性地融合了大语言模型服务,以 RAG 技术赋能系统的智能运维问答。

经过充分的黑盒与白盒测试以及高并发通信性能测试,系统的各项功能模块运作正常,满足设计初期的非功能性要求,具备高可用性和良好的扩展性,为私有云平台的低成本搭建和智能化运维提供了可靠的解决方案。

关键词:虚拟化云平台;Proxmox VE;Vue 3;Django;实验管理;大语言模型;RBAC

Comment by mecio: (1) 关键词字体黑体小三

(2) 关键词内容空格隔开

Abstract

With the widespread popularization of virtualization and cloud technologies, various industries have placed higher demands on the control of IT resources. Although the open-source platform Proxmox VE holds a significant position in the private cloud market, it still shows limitations when facing multi-

tenant isolation, deep business integration, and user UI design. To address this, this paper proposes and implements a cloud management platform built with the Vue 3 framework and a Django 5 backend. This research not only lowers the operational threshold of PVE clusters but also extends functionality for practical scenarios such as educational applications, enhancing the comprehensive management efficiency of the system.

The system follows the design philosophy of frontend-backend separation. The frontend is built based on the Vue 3 Composition API and the Vite build engine, utilizing the Vben Admin framework to create a high-performance data visualization management interface; meanwhile, it integrates LogicFlow for drawing network topologies and employs xterm.js and noVNC technologies to achieve immersive remote terminal interaction. The backend is constructed using Django 5 coupled with the DRF framework to build standardized RESTful interfaces, with MySQL used for underlying storage, and deep scheduling of physical resources completed through the PVE REST API and SSH protocols. Additionally, the system introduces the ASGI asynchronous architecture and WebSocket protocol to ensure the real-time nature and bidirectional communication quality of console data transmission.

The system focuses on implementing four core functions: first, full lifecycle management of virtual machines and container clusters, supporting basic monitoring and topological visualization of network resource allocation; second, the construction of an organizational structure and a multi-level fine-grained permission control system based on the RBAC model, ensuring platform security and providing interface-

level operation audit logs; third, collaborative management for dual roles in experimental teaching oriented towards educational scenarios, providing teachers and students with streaming distribution of online experimental environments and a Markdown-based experimental report editing experience; fourth, the innovative integration of Large Language Model (LLM) services, empowering the system's intelligent O&M (Operations and Maintenance) Q&A through RAG technology.

Following extensive black-box and white-box testing, as well as high-concurrency communication performance testing, all functional modules of the system operate normally and meet the non-functional requirements established at the initial design stage. The system possesses high availability and good scalability, providing a reliable solution for the low-cost construction and intelligent O&M of private cloud platforms. With the rapid development of cloud computing and virtualization technologies, the demand for refined and intelligent management of IT infrastructure in various industries is increasing. Proxmox VE (PVE), as an open-source and powerful enterprise-

level virtualization platform, has been widely applied in the construction of private clouds for small and medium-sized enterprises and educational institutions. However, the native PVE platform still has certain limitations in multi-tenant models, complex permission management, deep business scenarios (such as teaching experiments), and the modern presentation of user experiences. Therefore, this paper designs and implements a PVE virtualization cloud management platform based on Vue 3 and Django 5, aiming to lower the operation and maintenance threshold of PVE clusters and expand its application capabilities in vertical scenarios such as educational administration.

The platform is developed using a modern architecture with separated front-end and back-end. At the front-end level, based on the composition API of Vue 3, the Vite build mechanism, and the Vben Admin framework, a responsive and data-visualized operation management console is constructed. It deeply integrates the LogicFlow network topology tool, xterm.js, and noVNC to provide immersive terminal control interactions. At the back-end level, Django 5 and DRF are used to build RESTful APIs, combined with a MySQL relational database to complete data persistence. Deep scheduling of the infrastructure is achieved through the PVE REST API and SSH. Meanwhile, the ASGI asynchronous server and WebSocket protocol are adopted to realize real-time connection and two-way communication with instance consoles.

The system focuses on four core functions: First, the full life-cycle management of virtual machine and container clusters, supporting basic monitoring and topological visual allocation of network resources; Second, the construction of an organizational structure and a multi-level fine-grained permission control system based on the RBAC model, ensuring platform security and equipped with interface-level operation audit pipelines; Third, the dual-role collaborative management of experimental teaching for educational scenarios, providing teachers and students with streaming distribution of online experimental environments and a Markdown experimental report editing experience; Fourth, the innovative integration of large language model services, empowering intelligent operation and maintenance Q&A of the system with RAG technology.

Through sufficient black-box and white-box testing, as well as high-concurrency communication performance testing, the various functional modules of the system operate normally, meeting the non-functional requirements at the initial stage of design. It possesses high availability and good scalability, providing a reliable solution for the low-cost construction and intelligent operation and maintenance of private cloud platforms.

Keywords: Virtualization Cloud Platform; Proxmox VE; Vue 3; Django; Experiment Management; Large Language Model; RBAC

绪论

1.1 研究背景与意义

随着信息技术的快速发展,传统物理服务器因在资源利用率、运维成本、扩展弹性等方面的缺陷,无法满足现代化高并发应用与业务需求。云计算与虚拟化技术凭借高可用性、弹性扩展、资源池化等优势,已成为现代IT基础设施的核心支撑。然而,对于中小微企业和高校实验实训环境而言,普遍面临IT预算有限、专业运维人员不足、硬件资源相对匮乏、实验环境需要快速创建与回收等现实约束。通用的企业级私有云方案往往部署复杂、资源消耗大,难以直接落地。针对上述场景,设计一套轻量化、易管控、具备自服务能力的私有云集群管理平台(基于PVE),具有重要的现实应用价值。

在多元化的开源虚拟化选型中,基于 Debian 深度定制的企业级平台 Proxmox VE (PVE) 影响力日益增强。该平台原生整合了 KVM 全虚拟化与 LXC 容器化技术,并提供了功能完备的 Web 管理端,极大地简化了集群调度、网络布局及分布式

存储的部署流程。尽管如此,原生控制台的设计逻辑更偏向于底层运维与专业技术人员,其交互重心在于基础设施的物理属性,这使得它在满足非专业用户需求或适配特定垂直业务逻辑时,存在明显的体验门槛与功能适配缺口在众多开源的虚拟化解决方案中,Proxmox Virtual Environment (PVE) 作为一款基于 Debian Linux 开发的开源企业级服务器虚拟化环境,正越来越多地受到业界的青睐。PVE 内置集成了 KVM (Kernel-based Virtual Machine) 全虚拟化技术以及 LXC (Linux Containers) 轻量级操作系统级虚拟化机制,其自身也带有相对完善的 Web 界面,为集群资源分配、网络配置和分布式存储池的挂载提供了极大的便利。然而,PVE 原生的 Web 控制台在设计初衷上主要服务于底层的系统管理员或极客群体,侧重于展现基础设施的硬装调度状态,导致其在面向普通用户或是特定垂直应用场景时显得力不从心。

基于上述背景与面临的实际痛点,本文提出并完整设计与实现了一套基于现代 Web 技术栈 (Vue 3 + Django 5) 的 PVE 虚拟化云管理系统平台。本系统的研发具有直接的技术理论意义与深远的业务落地价值:一方面,在技术层面,通过使用前后端分离的现代化架构,重新重构和封装了原本生涩杂乱的 PVE 底层 API,并借助 WebSockets 和 ASGI 异步通信层使得终端交互达到了流畅的实时响应级别,极大降低了用户对 PVE 集群的操作门槛与学习成本;另一方面,在场景融合层面,本系统从传统的纯粹资源管控跨越到了完整的教务业务闭环中,融合实验的派发与报告回收流,并创新性地将基于 RAG (检索增强生成) 技术的大语言模型助手纳入平台支持,使得复杂的虚拟化集群系统演变为一个平易近人、具备“自驱动”问答能力的智能运维及实验辅助工作站。 Comment by mecio: 不用换行。

1.2 国内外虚拟化管理平台的研究现状

放眼全球云基础设施的软件研发历程,虚拟化平台及配套操作系统已实现跨越式发展,其技术重心已从孤立节点的虚拟化转型为大规模分布式环境下的协同控制在全球云计算基础软件的研发赛道中,虚拟化平台及云管操作系统的演进经历了从单节点资源虚拟化向超大规模分布式集群控制的发展。

目前,国外主流的商业化虚拟化解决方案首推 VMware 体系下的 vSphere 与 vCenter 平台 [https://www.vmware.com/products/cloud-infrastructure/vsphere]。该系列平台凭借其成熟稳定的 High Availability (HA) 和 Distributed Resource Scheduler (DRS) 能力,在政企级的公有云和私有云市场中处于垄断性地位。然而,VMware 系产品的封闭生态和高昂的商业授权费用,对于诸多预算有限的中小企业、科研团队和高校教育网络而言,形成了难以逾越的壁垒。在开源领域,OpenStack 项目作为最具代表性的基础设施即服务 (IaaS) 云操作系统,虽实现了计算、存储、网络的全面模块化治理,并得到大量互联网巨头的背书,但其架构庞杂、安装部署困难、后期运维的人力成本极高等弱点,让它只能成为大型数据中心的专属“奢侈品”。 Comment by mecio: 引用 Comment by mecio:

随着轻量级虚拟化和容器化技术 (特别是 Docker 与 Kubernetes 生态) 的兴起,一批具有轻量化管理特性的控制面板也迎来了快速演进时期。例如 Proxmox VE、oVirt 平台,亦或是完全基于轻量级 libvirt 的 WebVirtMgr 等边缘节点集群工具。它们能够非常优秀地撑起中小规模业务系统的底层虚拟机编排。但是,将这些偏重底层功能的平台直接作为企业对内、乃至对不同权限角色的终端服务门户使用时,往往显得水土不服。 Comment by mecio: 引用

纵观国内的研究机构与企业级研发团队,在针对国产替代及自身特定业务开展私有云定制化研发方面,取得了不少极具突破性的成果。许多互联网企业开始利用 Spring Boot (Java) 或是 Python 全栈框架配合前端 React/Vue 技术体系重写底层的虚拟化资源调度 API,将自身的工单审批流、计费策略或堡垒机审计系统与 KVM 底层深度嵌套。

Comment by mecio: 这个是否有一些知名的框架,如有,可以加上引用。

1.3 主要研究内容与目标

本研本课题的最终目的是依托当前成熟、现代化的 Web 前后端技术框架 (Vue 3 前端应用生态与 Django 5 复合后端生态),打造一款高性能、高可用且体验顺畅的基于 PVE 底层架构的上层云管理交互系统。本系统不再仅限于简单的开机重启,而是志在打造一个集实时资源监控、集群精细拆分调度、全周期实验教务管理流,加上大文件模型 AI 智能辅助的高阶综合云管解决方案平台。

前后端解耦的轻量级虚拟管理系统架构设计选型与验证

分析论证该系统中选用的各类开源框架和包的效能,前端使用 Vue 3 的 Composition API 和 Pinia 状态管理库以及 Vite 快速构建工具;后端的基座由 Django 5 和 Django REST Framework (DRF) 搭建,并在关系型数据库模型基础上结合 Redis 构建一套具有异步响应能力 (依托 Daphne、Channels) 的架构拓扑,实现轻量级 API 的无状态调用和全双工协议的长连接并存体系。 Comment by mecio: 用上述小标题进行分点描述。每点内容不宜过长50-100字。

基于官方 PVE API 与系统 SSH 的网络底层深度映射与抽象

研究并调用 PVE 层提供的原生 REST API 组件 (基于 ticket/CSRFToken 机制),把复杂的集群层、节点层直至内部的具体配置数据,全部剥丝抽茧提取为主系统数据流。探索自动化调用系统网络创建 (Bridge、VLAN等) 底层协议方法,进而

借助 LogicFlow 图形库在前台还原为对用户直观的网络拓扑可交互设计器;同时使用 Paramiko 与 AsyncSSH 封装直连底层机器和 LXC 容器的会话流,使得 noVNC 与 xterm.js 前端嵌入具备毫秒级的刷新体验感知。

契合企业与学校治理模型的细粒度权限和安全审计体系建设

剖析多租户环境和跨组织维度的权限管理需求,设计并实现包含用户控制、角色继承与菜单动态映射体系在内的 RBAC 权限设计模型。为敏感系统操作(如启动销毁实例、用户资料更改等)从数据层注入权限作用范围(Data Scope),并在 Django Middleware 中间件层面加入严格的全部接口全局流量审计拦截日志系统,杜绝任何提权与越权安全隐患。

教务协作流的垂直业务重塑与在线协作实验扩展

跳脱出原有基础设施监控软件的发展窠臼,针对高校及培训机构的教育实训场景进行系统拓展设计。主要囊括两大子模块与三条业务流:针对教师角色的实验发布;针对学生角色的按需实操接入体系,并且利用集成化 Markdown 富文本支持的沉浸式在线实验报告编辑器,形成发布、操作、编辑、归档的教育全生命周期在线闭环。

AI 智能助手架构的引流与 RAG 知识库融合体验

紧跟大语言智能生成时代的浪潮,将大型模型的交互应用化作为运维创新的破局点进行融合配置研究。实现在前端界面建立常驻的流式对话及问答机制(Stream Websocket Connect),并在后端引入不同模型的 API 和额度配置管理系统(Quota 管理)。更进一步地探索 RAG(Retrieval-Augmented Generation)向量检索的实际运用能力手段,将原本系统内静态的文档大纲以及知识帮助体系,动态地运用于 AI 的情景语义补全阶段,使系统变身为一个“能说会道”且可以及时解决平台环境疑问的虚拟专家。

2.系统相关技术介绍

本系统采用前后端分离架构。前端基于 Vue 3、Vite、Pinia、Vben Admin、xterm.js、noVNC、LogicFlow 和 ECharts 构建;后端基于 Django 5、Django REST Framework、SimpleJWT、Django Channels 和 Daphne 构建,数据库使用 MySQL;底层通过封装 PVE REST API 与 Paramiko、AsyncSSH 等库实现虚拟化资源管理与终端交互。本章对上述技术的原理与特性进行介绍。

2.1 前端核心技术

2.1.1 Vue 3 与 Vite

Vue.js 是渐进式 JavaScript 框架,其核心设计理念是可以被逐步采用——既可以作为简单的视图层库引入,也可以配合周边生态构建完整的单页面应用。

Vite 是由 Vue 作者尤雨溪主导开发的下一代前端构建工具,其核心思路是在开发阶段完全基于浏览器原生支持的 ES 模块(ESM)进行按需编译。这使得开发服务器的冷启动时间可以控制在毫秒级。

2.1.2 组合式 API (Composition API) 与 Pinia

组合式 API 是 Vue 3 提供的一套全新的逻辑组织方式。允许开发者以函数为单位,将同一功能相关的响应式状态、计算属性和生命周期逻辑集中编写,并通过自定义 Hooks(即以 use 为前缀的可复用函数)在多个组件间共享。这种方式避免了选项式 API 中同一业务逻辑被分散于多个配置块所导致的维护困难,在大型应用中尤为突出。组合式 API 同时也对 TypeScript 具有天然友好的类型推断支持。

Pinia 是 Vue 官方推荐的全局状态管理库,设计上比 Vuex 更为轻量 and 直观。Pinia 废弃了 Vuex 中 Mutation 的概念,将状态管理简化为 State(响应式状态)、Getters(派生状态)和 Actions(同步或异步操作)三个部分。每个 Store 以独立模块定义,天然支持代码分割;同时对 TypeScript 提供完整的类型推断,无需额外的类型声明即可获得准确的类型提示和编译期校验。

2.1.3 Ant Design Vue 与 Vben Admin

Ant Design 是由蚂蚁集团开源的企业级 UI 设计体系,其 Vue 版本实现为 Ant Design Vue,提供了涵盖布局、表单、表格、弹窗、图标等数十种高质量通用组件。组件遵循统一的设计规范,在视觉风格和交互行为上高度一致,适用于中后台管理系统的快速开发。

Vben Admin 是基于 Ant Design Vue 构建的开源中后台前端解决方案,在组件库之上进一步封装了路由管理、权限控制、动态菜单、布局骨架、主题配置等业务级基础设施,并集成了自动路由加载机制与完整的鉴权布局框架。开发者可以直接基于其提供的目录结构和约定进行业务开发,从而专注于具体功能实现而非底层架构搭建。

2.1.4 xterm.js、noVNC、LogicFlow 与 ECharts

xterm.js 是一种能让浏览器模拟真实命令行界面的工具。它能够完美识别各种终端指令,实现彩色文字显示、光标自由移动以及历史记录滚动等功能。借助其灵活的插件系统,用户还可以开启硬件加速或文本检索。当它与 WebSocket 技术结合

时,开发者可以直接在网页上建立一个反应敏捷、体验顺滑的远程操作窗口。xterm.js 是一个基于 Web 标准实现的全功能终端模拟器,运行于浏览器环境中。它支持完整的 VT100/VT220 ANSI 转义序列解析,能够正确渲染颜色、光标控制、滚动缓冲区等终端特性,并提供插件机制支持文本搜索、WebGL 加速渲染等扩展能力。通过与 WebSocket 配合,可以在浏览器中实现流畅的远程 Shell 交互。

作为一种纯 Web 化的 VNC 接入技术,noVNC 摆脱了传统插件的束缚,实现了远程桌面的浏览器原生访问。它通过 WebSocket 链路高效传输 RFB 协议帧,并交由 HTML5 的绘图引擎进行动态渲染。系统集成了完整的输入控制逻辑与剪贴板交互功能,确保了用户在 Web 环境下依然能获得流畅、闭环的桌面操控反馈noVNC 是一个纯 HTML5/JavaScript 实现的 VNC 客户端,无需安装任何本地软件即可在浏览器内访问远程桌面。它通过 WebSocket 将 RFB (Remote FrameBuffer) 协议帧传输至前端,利用 HTML5 Canvas 进行渲染,支持键盘鼠标的完整输入映射以及剪贴板同步。

LogicFlow 是由滴滴开源的流程图与拓扑图绘制框架,提供了可扩展的节点、边及交互行为定义体系。开发者可以通过其插件机制自定义节点外观与连线规则,支持拖拽连线、节点编辑、历史记录等交互能力,适合构建网络拓扑、流程设计等可视化场景。

Apache ECharts 是由 Apache 基金会托管的开源可视化图表库,基于 Canvas 和 SVG 双渲染引擎,支持折线图、柱状图、饼图、仪表盘、热力图等数十种图表类型。ECharts 在处理大规模数据时具有较好的渲染性能,并提供响应式布局、主题定制、数据动画以及流式数据更新等能力,广泛用于监控大屏和数据分析场景。

2.2 后端核心技术

2.2.1 Django 5 与 MySQL、Django ORM

Django 是基于 Python 的全栈 Web 框架,以"Batteries Included (内置电池)"著称,内置了路由、视图、模板、ORM、表单验证、缓存、认证、管理后台等完整的 Web 开发组件,开发者无需从零集成各类基础设施即可快速构建生产级应用。Django 5 在异步视图支持、数据库访问性能以及表单与模型 API 易用性等方面较前序版本均有所提升。

MySQL 是目前应用最为广泛的开源关系型数据库管理系统之一,以高稳定性、高并发读写性能和成熟的生态工具链著称,适用于存储具有明确结构和关联关系的业务数据。

Django ORM (Object-Relational Mapping,对象关系映射) 是 Django 内置的数据库抽象层。开发者以 Python 类定义数据模型,ORM 负责将其映射为数据库表结构,并将 Python 对象的操作自动转换为参数化 SQL 语句执行。参数化查询在底层彻底隔离了 SQL 指令与数据,从机制上杜绝了 SQL 注入风险。ORM 同时支持跨表关联查询、多对多关系、自引用层级结构等复杂模型的简洁表达。

2.2.2 RESTful 规范与 Django REST Framework

为了确保接口的兼容性与易用性,本研究遵循 RESTful 设计准则。其基本逻辑在于将业务实体抽象为 URI 标识的资源,并借助标准化的 HTTP 请求方法执行增删改查等动作。通过统一响应数据结构 (如 JSON),该风格不仅简化了前后端的协作流程,还显著增强了 API 在不同技术栈之间的集成效率REST (Representational State Transfer) 是一种面向资源的 Web API 设计风格,其核心约束包括:以 URI 唯一标识资源,以标准 HTTP 动词 (GET、POST、PUT、PATCH、DELETE) 表达操作语义,以及使用统一的数据格式 (通常为 JSON) 进行响应。遵循 RESTful 规范的 API 具有良好的可预测性和跨平台兼容性,便于前后端协作和第三方集成。

Django REST Framework (DRF) 是 Django 生态中最成熟的 RESTful API 构建扩展,提供了序列化器 (Serializer)、视图集 (ViewSet)、路由自动生成、权限控制、限流、过滤、分页等完整的 API 开发组件。其中序列化器承担请求数据的字段级验证、嵌套关联数据的解析以及查询结果到 JSON 格式的序列化转换,是 DRF 中处理数据进出的核心组件。

2.2.3 JWT 无状态认证与 RBAC 权限模型

JSON Web Token (JWT) 是一种基于 JSON 的开放标准 (RFC 7519),用于在通信双方之间以紧凑、自包含的方式传递声明 (Claims)。一个标准 JWT 由头部 (Header)、载荷 (Payload) 和签名 (Signature) 三部分组成,服务端通过验证签名即可确认令牌的合法性,无需查询服务器端存储的会话状态,因而具有天然的无状态 (Stateless) 特性。在前后端分离架构中,JWT 通常以 Bearer Token 形式附加于请求头,支持跨域资源共享 (CORS) 场景。djangorestframework-simplejwt 是 DRF 生态中的 JWT 实现库,提供 Access Token (短期有效) 与 Refresh Token (长期有效) 的签发、验证与刷新机制。

RBAC (Role-Based Access Control,基于角色的访问控制) 是一种将权限分配与用户角色绑定的权限管理模型。其基

本结构为:用户被赋予一个或多个角色,角色关联一组权限资源,系统在处理请求时根据当前用户所持角色判断其是否具备对目标资源的访问权限。RBAC 将权限管理从用户层面上升到角色层面,在用户数量庞大、角色类型多样的场景下,相比直接对用户授权具有更低的维护成本和更清晰的权限结构。

2.2.4 ASGI 异步架构、Django Channels 与 Daphne

ASGI (Asynchronous Server Gateway Interface) 是 Python Web 生态中 WSGI 的异步继任规范,支持 HTTP/1.1、HTTP/2 以及 WebSocket 等长连接协议的并发处理。与 WSGI 的同步阻塞模型不同,ASGI 服务器基于异步事件循环处理请求,可在单进程内以协程方式并发处理大量 I/O 等待任务,而无需为每个连接分配独立线程,适用于需要维持长时间连接的实时通信场景。

— Django Channels 是 Django 官方提供的 ASGI 扩展,为 Django 引入了 WebSocket、长轮询等协议的原生支持。Channels 以 Consumer (消费者) 为单位处理连接生命周期事件,并通过 Channel Layer (频道层) 实现进程间的消息传递与广播,Channel Layer 通常以 Redis 作为消息后端。

作为 Django Channels 生态中的核心组件,Daphne 充当了高性能的 ASGI 服务网关。它具备处理多元化协议的能力,能够将接入的请求精准转发至后端的业务逻辑层。该服务器最大的优势在于实现了 HTTP 短连接与 WebSocket 长连接的同端口混合调度,为复杂实时应用的部署提供了极大的便利Daphne 是 Django Channels 官方配套的 ASGI 服务器,负责接收外部连接并将其按协议类型分发给 Django 应用层处理,支持 HTTP 与 WebSocket 的混合部署。

2.3 PVE REST API

Proxmox VE (PVE) 是基于 Debian Linux 的开源服务器虚拟化平台,支持 KVM 全虚拟化和 LXC 容器两种虚拟化技术,并提供集群管理、存储管理、网络配置、备份与高可用等完整功能。PVE 对外暴露一套基于 HTTPS 的 REST API (路径前缀为 /api2/json),覆盖了节点、虚拟机、容器、存储、网络、任务等所有管理操作,是实现第三方平台对接的标准接口。

PVE 身份验证体系主要依托票据 (Ticket) 机制实现:外部请求需先访问 /access/ticket 接口提交身份信息,从而换取具备时效约束的 PVEAuthCookie 以及防御跨站攻击的 CSRFPreventionToken。在后续的交互过程中,这两项凭证必须分别置于 Cookie 和请求头 (Header) 中以完成合法性校验。此外,平台还兼容 API Token 授权模式,允许针对特定 Token 划定颗粒度更细的权限边界,这一特性尤为适用于异构系统间的自动化集成与免密调用PVE API 的认证机制基于票据 (Ticket) 体系:调用方首先向 /api2/json/access/ticket 端点提交凭据,获取具有时效性的 PVEAuthCookie 会话票据和用于防止跨站请求伪造的 CSRFPreventionToken,后续请求须在 Cookie 和请求头中分别携带这两项凭证。此外,PVE 也支持以 API Token 的形式进行认证,可为 Token 绑定细化的权限范围,适合服务间自动化调用场景。

针对涉及大量 I/O 或网络消耗的任务,PVE 默认以 Task 形式在后台运行。通过 API 交互获取到任务追踪码 (UPID) 后,前端或后端服务需利用轮询技术对指定路径进行状态查询。这种设计分离了指令下发与实际执行的过程,使得平台能高效管理从克隆到迁移等一系列复杂的基础设施调度工作PVE 中的大多数耗时操作 (如克隆虚拟机、在线迁移、磁盘扩容等) 均以异步任务 (Task) 形式执行,API 调用后立即返回任务 ID (UPID),调用方需通过轮询 /api2/json/nodes/{node}/tasks/{upid}/status 端点获取任务执行结果。

2.4 Paramiko 与 AsyncSSH

作为一种构建加密通道的网络协议,SSH 有效解决了在开放网络上进行敏感操作的安全隐患。其核心机制结合了公钥加密与对称加密的优势:首先通过非对称手段确立信任关系并交换密钥,此后则利用对称算法对通信链路进行全量加固。这一特性使其在远程登录、系统调控以及自动化脚本执行等场景中得到了极其广泛的应用SSH (Secure Shell) 是一种加密网络协议,用于在不安全网络上建立安全的远程命令执行和数据传输通道。SSH 基于非对称加密完成身份认证与密钥交换,后续通信使用对称加密保护,广泛用于服务器远程管理。

在 Python 领域,Paramiko 是处理 SSH 协议交互的首选框架。它支持多元化的准入认证机制,并提供了高度封装的远程 Shell 控制与数据传输接口。该库采用同步执行逻辑,使得开发者能够以线性编程的方式完成复杂的跨主机运维逻辑,特别适用于需要高可靠性保障的系统初始化、配置同步及常规运维管理场景Paramiko 是 Python 中成熟度最高的 SSH 协议实现库,提供了 SSH 客户端与服务端的完整实现,支持密码认证、公钥认证、交互式 Shell、命令执行、SFTP 文件传输以及端口转发等功能。Paramiko 基于同步 I/O 模型,适合脚本化的批量运维操作场景。

作为同步库 Paramiko 的高性能替代方案,AsyncSSH 利用异步 I/O 技术实现了对 SSH 协议的非阻塞调用。这种架构设计使得系统在面对高频、高并发的远程交互需求时,表现出更优的 CPU 与内存利用率。由于其天然契合 asyncio 模型,该库可以无感知地融入基于 ASGI 的现代后端架构,为实时运维平台提供极速的远程指令下发与通道管理能力AsyncSSH 是基于 Python asyncio 异步框架实现的 SSH 客户端与服务端库,API 设计与 Paramiko 类似,但所有 I/O 操作均为非阻塞的协

程调用。在需要同时维护大量 SSH 连接的场景中,AsyncSSH 相较于同步模型的 Paramiko 具有更低的资源开销和更高的并发上限,且可以与 ASGI 框架的事件循环无缝协作。

3.需求分析

3.1 系统可行性分析

可行性分析旨在评估系统开发的必要性与可行性,论证其能否在合理的技术条件、运营成本与用户接受度下实现预期目标。下面从技术可行性、操作可行性、经济可行性三个方面展开分析。

3.1.1 技术可行性

在前端技术栈方面,Vue 3 与 Vben Admin 提供了成熟的单页应用开发框架,组件库丰富、生态完善;xterm.js 与 noVNC 均为业界广泛验证的 Web 终端与远程桌面方案,可直接在浏览器中实现 SSH 文字终端与 VNC 图形化远程桌面接入,无需安装任何客户端软件。

在后端技术栈方面,Django 5 与 Django REST Framework (DRF) 具备成熟的 RESTful API 快速开发能力;Django Channels可支持基于 ASGI 协议的 WebSocket 长连接,满足控制台代理和实时监控的高并发场景。

在底层资源调度层面,Proxmox VE 开放了详尽的 REST API 标准与开发手册,实现了对物理节点、实例全生命周期、存储及网络拓扑等核心维度的全面覆盖,为系统交互提供了确定的协议保障。同时,得益于 Python 强大的生态环境,用于构建 SSH 加密通道的 Paramiko 库,以及诸如 OpenAI、DeepSeek、Moonshot 等主流大模型的流式交互接口,均具备成熟的 SDK 支持,从而降低了异构模块集成与功能调用的技术门槛在底层虚拟化对接方面,Proxmox VE 提供了完整的 REST API 规范与开发者文档,覆盖节点管理、虚拟机生命周期、存储、网络及任务查询等所有必要操作,消除了底层通信的不确定性。此外,SSH 通道访问所需的 Paramiko 库,以及主流大语言模型的流式 API (如 OpenAI、深度求索、月之暗面等) 均有完善的 Python 生态支持,可直接集成调用。

3.1.2 操作可行性

原生的 Proxmox VE Web 控制台面向底层运维工程师设计,包含大量硬件抽象层术语 (如 LVM-Thin 池、块设备、Ceph 仲裁等),对非专业运维背景的用户而言学习曲线陡峭,误操作风险较高。

本系统重新设计了用户界面与交互流程,使用现代化响应式 Web 设计,提供直观的向导式表单、可视化的网络拓扑编辑器,以及完善的输入校验与操作提示。与此同时,系统基于细粒度的 RBAC 角色权限机制,对不同角色的用户展示不同的菜单与操作入口:管理员可管理集群节点和资源;教师可发布和批阅实验;学生只需进入实验工作台即可直接访问分配好的虚拟机终端,无需了解底层基础设施细节。这种"以角色业务为中心"的交互设计大幅降低了不同层次用户的操作门槛,操作可行性良好。

3.1.3 经济可行性

本系统所选用的核心技术栈全部为开源免费方案:Debian/Ubuntu 服务器操作系统、Proxmox VE 虚拟化平台、Vue 3 前端框架、Django 5 后端框架、MySQL 数据库,无需支付商业软件授权费用。相比于 VMware vSphere 等商业虚拟化平台随规模增长的高额许可费,本方案在授权成本上具有明显优势。

在部署成本方面,系统基于标准 x86 服务器硬件运行,可充分利用高校现有机房设备或企业退役测试机器,延长硬件使用寿命,降低初期投入。长期来看,LXC 容器的轻量化资源复用特性也可显著提升硬件算力的利用率。在经济可行性上具有较高性价比。

3.2 功能需求分析

在系统逻辑架构设计中,我们将整体功能解构为:综合监测仪表盘、底层系统管理、PVE 资源管控、教务实验支撑及 AI 交互服务。针对各功能板块的业务需求,具体描述如下经过调研分析,系统的功能模块分为:仪表盘模块、系统管理模块、PVE 资源管理模块、实验教学模块以及 AI 助手模块。各模块的具体功能需求描述如下:

3.2.1 仪表盘模块

仪表盘为用户登录后的首页,提供整个虚拟化集群的全局状态概览。系统需定时轮询 PVE API,在仪表盘上以图表形式实时展示信息:

3.2.2 系统管理模块

用户管理模块:该功能模块负责处理账号的增删改查及停用逻辑,维护包含身份凭证在内的核心数据。除支持个人资料的设置外,系统重点强化了用户与组织实体、角色权限的绑定功能,确保了管理维度的多维化与精细化用户管理:支持用户账号的创建、查询、修改与停用。用户基本信息包括用户名、姓名、邮箱及密码;支持用户头像与个人简介的维护;提供用户与角色、组织的关联管理入口。

角色管控:提供角色创建与参数自定义功能,核心配置项涵盖名称、标识码及多级数据权限 (如全局、部门内或个人)。系

支持角色与前端菜单、后端 API 的精细化绑定角色管理;支持角色的创建与配置,包括角色名称、编码、描述及数据范围(全部/本部门/本人)设置。每个角色可关联若干菜单项与 API 权限,同一用户可被授予多个角色,其最终权限为所有角色权限的并集。

菜单管理:支持对系统前端菜单树的增删改查,菜单项属性包括标题、路由路径、前端组件路径、图标及排序序号,支持多层父子嵌套关系。系统根据当前用户所持角色动态下发可访问的菜单树,实现前端页面的自动裁剪。

权限管理:该模块负责管理系统所有的接口访问权限,记录包含 URL 模式、方法类型及编码在内的关键参数。通过将 API 权限与菜单逻辑绑定,系统能够在执行层面实现接口级的鉴权拦截,保障了业务数据在权限架构下的安全性与隔离性支持对 API 级权限条目的维护,每条权限记录包含权限名称、权限编码、HTTP 方法、URL 匹配模式及关联菜单,用于在后端对每个接口端点进行细粒度的 RBAC 拦截校验。

组织管理:本功能负责维护多级递归的组织网络,支持复杂结构的层级映射。管理员除负责组织节点的基础维护外,还可管理用户与各级单元的隶属关系;通过主组织设定功能,系统实现了用户身份在多重组织背景下的唯一主权定义,优化了权限管理维度支持构建多层级树形组织架构(如"院系—班级"嵌套关系),支持组织的创建、编辑与停用,以及用户与组织单元的归属绑定,并支持为用户设置主组织。

操作日志:系统通过后端中间件自动拦截并记录所有经过认证的 API 请求,存储字段包括操作用户、请求路径、HTTP 方法、请求参数(脱敏)、响应状态码、客户端 IP 及 User-Agent、接口耗时等。操作日志仅供查询,不允许通过接口修改或删除,支持按时间范围、用户、操作类型、请求方法和状态码进行过滤检索。

登录审计日志:该模块专注于用户身份验证轨迹的留存,详尽记录了登录者的身份标识、网络地址、设备浏览器特征及交互时段。通过提供便捷的检索接口,管理人员可快速定位历史登录细节,从而为平台安全监管与隐患排查提供坚实的数据支撑登录日志:专门记录用户的登录与注销事件,存储登录用户名、IP 地址、User-Agent 及操作时间,支持检索查询,为安全审计提供登录行为的追溯依据。

3.2.3 PVE 资源管理模块

PVE 服务器管理:该功能覆盖了 KVM 实例从创建到销毁的完整周期。配置部署支持对核心硬件指标进行精细化设定;状态调度涵盖了重后、挂起、恢复及多种电源操作。针对高可用与数据安全需求,平台提供了虚拟机克隆、多节点调度以及快照的管理能力;其备份系统不仅支持全量备份与按需恢复,还提供了保护位设置及备注标注功能。支持添加、编辑与删除多台 PVE 物理服务器,提供一键连接测试功能。

虚拟机管理:提供 KVM 虚拟机的全生命周期管理能力。创建方面,支持通过向导式表单配置 CPU 核心数、内存大小、磁盘容量与存储池类型、网卡配置及操作系统类型;电源控制方面,支持启动、正常关机、强制关机、重后、挂起与从挂起中恢复操作;克隆方面,支持将虚拟机或模板克隆为新实例,可指定新 VMID 与目标节点;快照管理方面,支持快照的创建、查询、回滚与删除;备份管理方面,支持备份的创建、列表查询、从备份恢复及删除,支持备份备注与保护标志设置;此外,支持修改 CPU/内存等硬件配置参数,以及删除操作。

存储管理:该功能专注于节点级存储实体的状态感知。通过调用底层接口,系统可罗列所有在线存储池,并对存储容量的使用分布进行精准映射,确保管理员能够直观监测存储负载情况支持查询指定 PVE 节点下的所有存储池列表及各存储的容量使用情况。

PVE 节点监控:提供PVE节点的详细监控视图,除仪表盘已有的实时状态数据外,还支持资源走势曲线以及基于多维阈值的健康告警列表等。

模板管理:复用存储管理的内容浏览接口,专门用于查看和维护各存储中的操作系统克隆模板与 ISO 镜像资源,支持向存储上传新的模板文件。

网络管理:具备对 PVE 节点物理网卡与虚拟网桥(Bridge)的全面检测能力。该功能可直观映射接口的技术类型、逻辑绑定关系以及当前的链路实时状态,旨在协助管理人员清晰掌握底层节点的网络拓扑架构与配置详情支持查询 PVE 节点上的所有物理网络接口及虚拟网桥信息,展示接口类型、关联关系及活跃状态,为管理员提供节点网络架构的查看依据。

网络拓扑:提供基于图形化界面的拓扑设计工具。管理员能够自由定义节点属性并建立逻辑连接,以映射实际的物理与虚拟网络环境;系统支持拓扑数据的结构化存储,使用户可以随时调取历史配置进行维护或架构调整提供可交互的自定义网络拓扑图编辑功能。管理员可在前端画布上自由拖拽添加节点与连线,构建反映实际网络架构的拓扑示意图,并将拓扑数据持久化保存至数据库,支持后续加载与修改。

拓扑管理:该功能实现了网络架构的图形化复现。利用前端画布的交互机制,用户可手动组装网络节点并绘制关联线条,生成的拓扑关系直接持久化至数据库中。这不仅为网络架构提供了直观的展示窗口,也为后续的配置更新与状态追踪提供了修改入

口提供与网络拓扑模块联动的拓扑视图管理能力,支持对网络拓扑结构进行信息维护以及预览。

__LXC 容器管理:提供 LXC 轻量级容器的全生命周期管理。功能与虚拟机管理类似,支持容器的创建、电源操作、克隆与删除;提供基于 WebSocket + xterm.js 的 SSH 终端,使用户无需任何客户端即可直接在浏览器中与容器进行交互式命令行操作。

3.2.4 实验教学模块

实验管理:支持创建实验课程,填写课程编号、名称、描述、实验开放时间、截止时间及评分标准。实验遵循草稿→已发布→已归档的状态流转,发布前须上传指导书并设置评分标准。支持向实验上传多格式指导书文件,并提供在线预览与下载功能;文件上传后自动触发后台异步向量化索引任务,为 AI 知识库问答提供语料。支持将全部学生的提交成绩导出为 Excel 文件,包含学号、姓名、得分、是否逾期及提交状态等字段。

__提交管理:教师可查看所有学生对各实验的提交列表,支持按实验、提交状态(草稿/已提交/已批阅)、批改状态及是否逾期进行过滤。对于状态为"已提交"的记录,教师可录入得分、整体评语及详细评分项,保存后提交状态变更为"已批阅",并记录批改时间与批改教师;如报告存在质量问题,教师可将其退回至草稿状态并附上退回原因,学生可在修改后重新提交,系统记录累计退回次数。

我的实验:学生登录后进入实验工作台,仅展示当前已发布且处于激活状态的实验列表,屏蔽与学生无关的管理功能。学生进入某一实验后,系统自动为其创建草稿状态的提交记录;学生在内置 Markdown 富文本编辑器中撰写实验报告,支持上传截图等附件,支持随时保存草稿,最终确认无误后一键提交,系统记录提交时间并判断是否逾期。

3.2.5 AI 助手模块

仪表盘:展示 AI 功能的使用统计数据,包括总对话数、消息数、Token 消耗量及各模型的调用分布情况,供管理员掌握整体使用态势。

__模型管理:管理员可配置多个大语言模型接入端点,配置项包括服务提供商、API 基础地址、模型名称、温度参数及最大 Token 数等;支持独立启用或停用某一模型配置,使系统可灵活切换不同模型服务而无需修改代码。

API 管理:持对各厂商模型 API Key 的集成化配置。管理员可实时维护密钥库,系统通过脱敏算法保障了管理界面的数据私密性。此外,该模块强调了密钥滚动更新能力,支持在不中断业务逻辑的前提下完成 Key 的平滑切换,确保了 AI 接入层的合规性与安全性;对各模型配置所使用的 API Key 进行集中管理,支持添加、查看(Key 值脱敏展示)、轮换与删除;Key 轮换时生成新密钥并作废旧密钥,降低密钥泄露风险。

__额度管理:该模块允许对单一用户实施 Token 使用量的精细化定义。通过对每次请求的输入提示与生成结果进行分项计数,系统实现了对资源消耗的闭环监控,确保在用户达到预设阈值时即刻禁止新任务的提交,保障资源的合理分配;管理员可为每位用户单独配置 AI 使用配额(按 Token 数量),系统记录每次模型调用的实际 Token 消耗(含输入/输出分项),并在用户超出配额时拒绝新的调用请求。

__智能体管理:支持为特定实验课程绑定专属 AI Agent,配置其关联的模型、系统提示词(system prompt)及知识库范围,使不同实验可拥有各自定制化的问答助手角色与回答风格。

知识库管理:提供实验手册向量化进度的全局视图,实时监测索引状态(如就绪、处理中、异常等),并详尽记录文档切片总数与执行反馈。系统支持针对特定文档的手动重构逻辑,允许管理员强制同步索引数据,确保检索库内容与最新版本的指导书文件达成强一致性;展示各实验指导书的向量化索引状态(待处理/处理中/已完成/失败),包括分块数量与处理备注;支持管理员手动触发对指定文档的索引重建,以确保知识库数据与最新指导书文件保持同步。

3.3 非功能性需求分析

非功能性需求是指系统在满足业务功能之外,对系统运行质量、安全性、可靠性及可维护性等方面的约束与要求。本系统的典型非功能性需求主要包含安全性需求和可维护性需求。

3.3.1 安全性需求

为保证系统的安全性需求,所有前后端通信须通过 HTTPS/TLS 协议加密,WebSocket 连接须使用 WSS 协议,防止传输层数据窃听与篡改。系统采用 JWT 令牌进行身份验证,令牌具有过期时间,支持刷新机制;每个 API 端点均须经过 RBAC 权限校验,未授权访问返回 403 状态码。控制台会话令牌设计为一次性使用,有效期不超过 60 秒,防止令牌重放。

PVE API Token、AI API Key 等敏感凭证不得在前端响应中明文暴露。系统通过 Django ORM 进行数据库操作,规避 SQL 注入风险,并防御 CSRF 与 XSS 等常见 Web 攻击。

3.3.2 可维护性需求

为增强系统的可维护性,后端按功能划分为独立的 Django App (rbac、pve、experiments、chat、audit、tasks、

system) ,各模块职责单一、低耦合,便于独立迭代与替换。可变的业务配置参数 (如 AI 模型端点、功能开关等) 通过 SystemSetting 模型存储于数据库,支持管理员在不重启服务的情况下动态修改。后端 API 遵循 RESTful 设计规范,具备统一的分页、过滤、搜索与排序参数格式,降低前后端联调成本与后期维护难度。

4.系统设计

本章将从系统设计原则、系统总体设计与数据库设计三个层次,对基于 PVE 的虚拟化云管平台进行系统化的设计论述。

4.1 系统设计原则

为保证系统具备良好的可维护性、可扩展性与安全性,本系统在设计阶段遵循以下核心原则:

(1) 前后端分离原则:系统采用前后端分离架构,前端基于 Vue 3 框架构建单页应用 (SPA) ,后端基于 Django 5 框架提供 RESTful API 服务。前后端通过标准化 JSON 接口解耦,可独立开发、独立部署,互不干扰,提升了并行开发效率与后期维护的灵活性。

(2) 模块化设计原则:系统在前端按功能域划分独立路由模块,在后端按业务领域划分独立 Django App (rbac、pve、experiments、chat、audit 等) 。各模块职责单一、内聚度高,模块间通过接口依赖而非实现依赖,降低耦合度,便于功能迭代与替换。

(3) 安全性优先原则:系统采用 JWT 令牌机制进行身份认证,所有需要鉴权的 API 端点均需经过 RBAC (基于角色的访问控制) 权限校验。WebSocket 控制台通道使用一次性临时票据 (Ticket) 防止令牌重放。敏感配置 (如 PVE API Token、AI API Key) 在数据库中加密存储,禁止在前端响应中明文返回。

(4) 接口标准化原则:后端所有 API 遵循统一的请求规范 (分页、过滤、排序参数格式一致) 与响应格式 (统一包装层,含 code、message、data 三字段) ,降低前后端联调成本,方便前端进行全局请求拦截与错误处理。

(5) 用户体验优先原则:面向非专业运维用户,系统对复杂底层操作 (如 PVE 克隆、快照、迁移等) 进行图形化封装,提供向导式表单、可视化拓扑编辑器与实时状态反馈,降低操作门槛,减少误操作风险。

4.2 系统总体设计

4.2.1 系统架构设计

本系统采用三层前后端分离架构,自上而下分为前端交互展现层、后端业务代理层与底层基础设施层,各层通过规范化接口通信,职责清晰、耦合度低。系统架构设计如图 4.1 所示。

图 4.1 系统架构设计图

前端交互展现层负责向用户呈现可视化界面并响应操作指令。用户在浏览器中通过 Vue 3 渲染的单页应用访问系统,界面框架采用 Vben Admin,组件库选用 Ant Design Vue。数据可视化 (仪表盘统计图表) 使用 Apache ECharts,网络拓扑可视化采用 Cytoscape.js,LXC 容器 SSH 终端使用 xterm.js,虚拟机图形控制台使用 noVNC (RFB 协议) 。前端通过 Axios 处理常规 HTTP 请求,通过 WebSocket 处理 VNC 图形流、SSH 终端代理及 AI 流式回复等长连接场景。

系统核心业务逻辑由基于 Django 5 的后端代理层驱动。在 Daphne 服务器的支撑下,该层级能够并发处理多种协议请求,并利用 SimpleJWT 令牌与基于角色的访问控制 (RBAC) 确保接口安全性。业务流转逻辑如下:涉及本地数据的操作通过 Django ORM 进行持久化;涉及虚拟化环境的指令则由专用的异步代理模块转换为 PVE 原生接口调用。该模块不仅负责 Ticket 和 CSRF Token 的动态映射,还承担了对大规模异步操作 (如虚拟机克隆) 的执行轨迹追踪职能。后端业务代理层是整个系统的核心枢纽,基于 Django 5 + Django REST Framework 构建,通过 Daphne + Django Channels 支持 ASGI 协议,同时处理 HTTP 与 WebSocket 请求。所有请求经过 SimpleJWT 令牌校验与 RBAC 权限拦截后,分流至对应业务模块:上层业务操作 (用户、实验、AI 配置等) 通过 Django ORM 直接与 MySQL 交互;底层 PVE 集群操作则由内置的 PVE Async Proxy 模块将请求转换为携带 Ticket/CSRF Token 的 PVE 原生 REST 请求,代理提交至 PVE 集群执行,并负责长耗时异步任务的追踪。

底层基础设施层位于系统内网隔离区域,包括 Proxmox VE 集群 (提供 KVM 虚拟机与 LXC 容器算力) 、MySQL 数据库 (持久化业务数据) 、Redis (作为 Django Channels 的 Channel Layer 与 Celery 消息队列) 以及可选的本地大语言模型服务节点,不对公网直接暴露任何端口。

4.2.2 系统功能设计

根据第三章的功能需求分析,本系统共设计五大功能模块:仪表盘模块、系统管理模块、PVE 资源管理模块、实验教学模块与 AI 助手模块。系统功能设计如图 4.2 所示。

图 4.2 系统功能模块设计图 Comment by mecio: Comment by mecio: 这个图的底色建议去掉,还有灰色的格子线也不要,最好是导出的png图,而不是屏幕截图,上面哪个架构图也是。

每个功能模块的设计内容如下:

(1) 仪表盘模块设计

仪表盘是用户登录后的首页,提供平台全局状态的聚合概览。页面采用响应式栅格布局,分为三个区域:顶部展示 4 张统计卡片(PVE 在线节点数、本月 AI Token 消耗量、实验项目总数、用户总数);中部左侧为 ECharts 折线/柱状双轴组合图展示近 14 天 AI Token 消耗趋势,右侧上方为三个圆形进度条展示主节点 CPU/内存/磁盘使用率(颜色随阈值动态变化:绿<75%、橙<90%、红≥90%),右侧下方展示最近实验列表;底部左侧展示 ISO 镜像与 LXC 模板列表,右侧展示 AI 各模型用量占比的环形饼图。所有数据在页面加载时通过 Promise.allSettled 并发请求,任意接口异常不影响其他数据展示。
Comment by mecio: 像这样关于界面的设计,可以简单画一个平面设计图。

(2) 系统管理模块设计

系统管理模块承载平台的基础运维管理功能。用户管理支持账号增删改查,以及与角色、组织的关联绑定;角色管理支持角色创建与权限配置,每个角色可关联若干菜单项与 API 权限,同一用户可被授予多个角色,最终权限取并集;菜单管理维护前端菜单树(支持多层父子嵌套),系统根据当前用户角色动态下发可访问的菜单树,实现前端页面的自动裁剪;权限管理维护 API 级别的权限条目,每条记录包含 HTTP 方法与 URL 正则匹配规则,用于后端接口细粒度拦截;组织管理构建多层级树形组织架构;操作日志通过后端中间件自动记录,仅读不写;系统设置以键值对形式存储可变业务配置,支持不重启服务动态修改。

(3) PVE 资源管理模块设计 Comment by mecio: 和上面对齐一下。

PVE 资源管理模块是系统的核心功能区,通过 PVE Async Proxy 代理层与 Proxmox VE 集群对接。服务器管理维护多台 PVE 物理服务器的连接配置,支持一键连通性测试。虚拟机管理以表格形式展示所有 KVM 虚拟机,支持创建、电源操作及需二次输入 VMID 确认的危险删除操作;虚拟机详情页采用左侧竖向 Tab 导航,包含概要、控制台(noVNC 图形桌面)、硬件配置、选项、备份管理与快照管理六个子页面。LXC 容器管理功能结构与虚拟机类似,控制台子页面提供基于 xterm.js 渲染的 WebSSH 终端,通过 WebSocket 与后端 Paramiko SSH 代理通信,支持字体、多套主题等个性化配置。网络拓扑包含"实时拓扑"与"拓扑设计"两个 Tab:实时拓扑使用 Cytoscape.js 自动布局渲染从 PVE API 获取的节点与网络连接关系;拓扑设计提供基于 Cytoscape.js 的交互式画布,管理员可自由拖拽节点与连线,构建并持久化自定义网络拓扑图。

(4) 实验教学模块设计

实验教学模块面向教师和学生两类角色提供差异化视图。教师端实验管理页采用卡片式布局,支持按标题搜索及按状态过滤,每张卡片展示实验基本信息、提交率/批改率/总分统计及操作按钮区(详情、批改、发布/归档、编辑、成绩导出、删除);实验创建/编辑表单支持上传多格式指导书文件,文件上传后后台自动触发向量化索引任务;提交管理页供教师查看学生提交记录并进行批改,支持退回操作。学生端仅展示已发布的实验列表;报告编辑器页采用左右分栏布局(左侧指导书预览,右侧基于 marked.js 的 Markdown 编辑区),支持附件上传与草稿保存,最终一键提交后系统自动记录时间并判断是否逾期。

(5) AI 助手模块设计

AI 助手模块管理平台内集成的大语言模型服务。模型管理支持配置多个大语言模型接入端点,可随时启用/停用指定模型;API Key 管理对 AI API Key 进行集中脱敏管理;额度管理为每位用户单独配置 Token 用量配额;智能体管理为特定实验课程绑定专属 AI Agent,配置关联模型与系统提示词(System Prompt),使不同实验拥有定制化的问答助手;知识库管理展示实验指导书的向量化索引状态,支持管理员手动触发索引重建。用户侧通过浮动 AI 聊天窗口与智能体对话,后端通过 WebSocket 将大语言模型的逐 Token 输出实时推送至前端,实现流式打字效果。

4.3 系统数据库设计

系统采用 MySQL 作为关系型数据库,存储平台业务层数据(用户权限、实验教学、AI 配置等)。PVE 集群内部的虚拟机实时运行状态以代理方式从 PVE API 获取,不做全量持久化。系统主要数据库表如表 4.1 所示。

表 4.1 数据库表汇总列表

序号	表名	所属模块	说明
1	auth_user	系统管理	用户信息表
2	rbac_userprofile	系统管理	用户扩展信息表(头像、简介)
3	rbac_role	系统管理	角色表
4	rbac_menu	系统管理	菜单表
5	rbac_permission	系统管理	API 权限表
6	rbac_organization	系统管理	组织架构表
7	rbac_userrole	系统管理	用户角色关联表

- 8 rbac_role_menus 系统管理 角色菜单关联表
 - 9 rbac_role_permissions 系统管理 角色权限关联表
 - 10 rbac_userorganization 系统管理 用户组织关联表
 - 11 audit_operationlog 系统管理 操作日志表
 - 12 system_systemsetting 系统管理 系统配置键值表
 - 13 tasks_job 系统管理 定时任务配置表
 - 14 pve_pveserver PVE 管理 PVE 服务器配置表
 - 15 pve_virtualmachine PVE 管理 虚拟机信息表
 - 16 pve_lxccontainer PVE 管理 LXC 容器信息表
 - 17 pve_networktopology PVE 管理 网络拓扑存储表
 - 18 experiments_courseexperiment 实验教学 实验课程表
 - 19 experiments_courseguidebook 实验教学 实验指导书表
 - 20 experiments_coursesubmission 实验教学 学生提交记录表
 - 21 experiments_courseattachment 实验教学 提交附件表
 - 22 chat_aimodelconfig AI 助手 AI 模型配置表
 - 23 chat_aiaapikey AI 助手 AI API Key 表
 - 24 chat_aiauserquota AI 助手 用户 AI 配额表
 - 25 chat_aiaagentconfig AI 助手 AI 智能体配置表
 - 26 chat_chatconversation AI 助手 对话会话表
 - 27 chat_chatmessage AI 助手 对话消息表
 - 28 chat_aiusagelog AI 助手 AI 调用记录表
- 4.3.1 数据字典设计 Comment by mecio: 格式对齐
- auth_user 表（用户信息表）

用户信息表是整个权限管理体系的基础,存储平台所有账号的核心身份信息,通过与角色表的关联关系确定每位用户的操作边界。具体设计如表 4.2 所示。

表 4.2 auth_user 表 Comment by mecio: Comment by mecio: Comment by mecio:

序号	名称	类型	不为空	注释
1	id	int	√	用户 ID,自增主键
2	username	varchar (150)	√	登录用户名（唯一,可为学号或工号）
3	password	varchar (128)	√	登录密码（PBKDF2 哈希加盐）
4	first_name	varchar (150)	√	名字
5	last_name	varchar (150)	√	姓氏
6	email	varchar (254)	√	邮箱地址
7	is_staff	tinyint (1)	√	是否为后台管理员
8	is_active	tinyint (1)	√	账号是否启用（0=停用）
9	is_superuser	tinyint (1)	√	是否为超级管理员
10	date_joined	datetime	√	账号创建时间
11	last_login	datetime		最后登录时间

audit_operationlog 表（操作日志表）

操作日志表由后端中间件自动写入,记录所有通过鉴权的 API 操作行为,为安全审计提供不可篡改的行为凭据。该表仅允许追加写入,不提供通过接口修改或删除的能力。具体设计如表 4.8 所示。

表 4.8 audit_operationlog 表

序号	名称	类型	不为空	注释
1	id	bigint	√	日志 ID,自增主键
2	user_id	int		操作者 ID（外键 → auth_user）
3	username	varchar (150)	√	操作者用户名（冗余存储）

- 4 action_type varchar (20) √ 操作类型 (CREATE/UPDATE/DELETE 等)
 - 5 request_path varchar (500) √ 请求 URL 路径
 - 6 request_method varchar (10) √ HTTP 方法
 - 7 request_params json √ 请求参数快照 (敏感字段脱敏)
 - 8 ip_address char (39) 客户端 IP 地址
 - 9 user_agent varchar (500) √ 客户端 User-Agent
 - 10 status_code int HTTP 响应状态码
 - 11 object_repr varchar (255) √ 操作对象描述
 - 12 created_at datetime √ 操作时间
- pve_pveserver 表 (PVE 服务器配置表)

PVE 服务器配置表存储受系统管辖的 Proxmox VE 宿主机连接参数,认证 Token 在数据库中加密存储,API 响应时不明文返回。具体设计如表 4.9 所示。

表 4.9 pve_pveserver 表

- | 序号 | 名称 | 类型 | 不为空 | 注释 |
|----|--------------|---------------|-----|-----------------------------|
| 1 | id | bigint | √ | 服务器 ID,自增主键 |
| 2 | name | varchar (100) | √ | 服务器名称 |
| 3 | host | varchar (255) | √ | 服务器 IP 或域名 |
| 4 | port | int | √ | API 端口 (默认 8006) |
| 5 | token_id | varchar (100) | √ | PVE API Token ID |
| 6 | token_secret | varchar (255) | √ | PVE API Token Secret (加密存储) |
| 7 | verify_ssl | tinyint (1) | √ | 是否验证 SSL 证书 |
| 8 | is_active | tinyint (1) | √ | 是否启用 |
| 9 | created_at | datetime | √ | 创建时间 |

pve_virtualmachine 表 (虚拟机信息表)

虚拟机信息表存储从 PVE 集群同步的 KVM 虚拟机记录,系统通过 server_id + vmid 的联合唯一键定位 PVE 集群中的具体实例。具体设计如表 4.10 所示。

表 4.10 pve_virtualmachine 表

- | 序号 | 名称 | 类型 | 不为空 | 注释 |
|----|------------|---------------|-----|---------------------------------|
| 1 | id | bigint | √ | 系统内部 ID,自增主键 |
| 2 | server_id | bigint | √ | 所属 PVE 服务器 (外键 → pve_pveserver) |
| 3 | vmid | int | √ | PVE 原生虚拟机 ID (与 server_id 联合唯一) |
| 4 | name | varchar (255) | √ | 虚拟机名称 |
| 5 | node | varchar (100) | √ | 所在 PVE 物理节点名称 |
| 6 | status | varchar (20) | √ | 运行状态 (running/stopped/paused) |
| 7 | cpu_cores | int | √ | 分配 CPU 核心数 |
| 8 | memory_mb | int | √ | 分配内存大小 (MB) |
| 9 | disk_gb | int | √ | 主磁盘容量 (GB) |
| 10 | ip_address | varchar (255) | √ | IP 地址 |
| 11 | pve_config | json | √ | PVE 原始配置 JSON 快照 |
| 12 | created_at | datetime | √ | 同步入库时间 |

experiments_courseexperiment 表 (实验课程表)

实验课程表是实验教学模块的核心实体,记录由教师创建的各类实验课程信息,通过 status 字段驱动草稿→发布→归档的完整生命周期。具体设计如表 4.11 所示。

表 4.11 experiments_courseexperiment 表

- | 序号 | 名称 | 类型 | 不为空 | 注释 |
|----|----|--------|-----|------------|
| 1 | id | bigint | √ | 实验 ID,自增主键 |

- 2 teacher_id int √ 创建教师（外键 → auth_user）
- 3 title varchar (200) √ 实验标题
- 4 course_code varchar (50) √ 课程代码
- 5 description longtext √ 实验描述
- 6 category varchar (50) √ 实验分类
- 7 difficulty varchar (20) √ 难度等级 (easy/medium/hard)
- 8 start_time datetime √ 实验开放时间
- 9 end_time datetime √ 实验截止时间
- 10 late_submission_allowed tinyint (1) √ 是否允许逾期提交
- 11 pve_template_id varchar (100) √ 关联 PVE 克隆模板标识
- 12 total_score decimal (5,2) 总分
- 13 scoring_criteria json √ 评分标准 (JSON 格式)
- 14 status varchar (20) √ 状态 (draft/published/archived)
- 15 is_active tinyint (1) √ 是否激活 (学生可见)
- 16 created_at datetime √ 创建时间

experiments_coursesubmission 表 (学生提交记录表)

学生提交记录表记录每位学生针对每个实验的报告提交状态与批改结果,通过 experiment_id + student_id 联合唯一约束保证每人每个实验只有一条提交记录。具体设计如表 4.12 所示。

表 4.12 experiments_coursesubmission 表

- | 序号 | 名称 | 类型 | 不为空 | 注释 |
|----|-------------------|---------------|-----|--|
| 1 | id | bigint | √ | 提交 ID,自增主键 |
| 2 | experiment_id | bigint | √ | 关联实验 (外键 → experiments_courseexperiment) |
| 3 | student_id | int | √ | 提交学生 (外键 → auth_user) |
| 4 | submission_status | varchar (20) | √ | 提交状态 (draft/submitted/graded/returned) |
| 5 | report_title | varchar (255) | √ | 报告标题 |
| 6 | report_content | longtext | √ | 报告正文 (Markdown 格式) |
| 7 | submit_time | datetime | | 提交时间 |
| 8 | is_late | tinyint (1) | √ | 是否逾期提交 |
| 9 | graded_by_id | int | | 批改教师 (外键 → auth_user) |
| 10 | score | decimal (5,2) | | 批改得分 |
| 11 | grade_status | varchar (20) | √ | 批改状态 (pending/graded) |
| 12 | feedback | longtext | √ | 教师评语 |
| 13 | scoring_details | json | √ | 各项评分明细 (JSON 格式) |
| 14 | revision_count | int | √ | 累计退回修改次数 |
| 15 | graded_at | datetime | | 批改完成时间 |

chat_aimodelconfig 表 (AI 模型配置表)

AI 模型配置表存储可对接的大语言模型接入端点,支持多模型并存配置,管理员可随时启用或停用指定模型而无需重启服务。具体设计如表 4.13 所示。

表 4.13 chat_aimodelconfig 表

- | 序号 | 名称 | 类型 | 不为空 | 注释 |
|----|------------|---------------|-----|----------------------------------|
| 1 | id | bigint | √ | 模型配置 ID,自增主键 |
| 2 | model_key | varchar (50) | √ | 模型唯一标识键 (唯一) |
| 3 | model_name | varchar (100) | √ | 模型名称 (如 gpt-4o、deepseek-chat) |
| 4 | model_type | varchar (50) | √ | 模型类型 (chat/embedding 等) |
| 5 | api_key_id | bigint | | 关联 API Key (外键 → chat_aiaapikey) |
| 6 | is_enabled | tinyint (1) | √ | 是否启用 |

- 7 is_default tinyint (1) √ 是否为默认模型
 - 8 max_tokens int √ 最大 Token 数限制
 - 9 temperature_default decimal (3,2) √ 默认温度参数
 - 10 allowed_roles json 允许访问的角色列表 (JSON)
 - 11 created_at datetime √ 创建时间
- pve_lxccontainer 表 (LXC 容器信息表)

LXC 容器信息表与虚拟机信息表结构类似,存储从 PVE 集群同步的 LXC 轻量级容器记录,额外包含 SSH 密码字段以支持后端建立 WebSSH 代理隧道。具体设计如表 4.20 所示。

表 4.20 pve_lxccontainer 表

序号	名称	类型	不为空	注释
1	id	bigint	√	系统内部 ID,自增主键
2	server_id	bigint	√	所属 PVE 服务器 (外键 → pve_pveserver)
3	vmid	int	√	PVE 原生容器 ID (与 server_id 联合唯一)
4	name	varchar (255)	√	容器名称
5	node	varchar (100)	√	所在 PVE 物理节点名称
6	status	varchar (20)	√	运行状态 (running/stopped)
7	cpu_cores	int	√	分配 CPU 核心数
8	memory_mb	int	√	分配内存大小 (MB)
9	disk_gb	int	√	主磁盘容量 (GB)
10	ip_address	varchar (255)	√	IP 地址
11	description	longtext	√	容器描述
12	pve_config	json	√	PVE 原始配置 JSON 快照
13	ssh_password	varchar (255)	√	SSH 密码 (用于 WebSSH 代理,加密存储)
14	created_at	datetime	√	同步入库时间

pve_networktopology 表 (网络拓扑存储表)

网络拓扑存储表持久化管理员在拓扑设计画布上创建的自定义网络拓扑图,拓扑节点与连线数据以 JSON 格式存储在 diagram_data 字段中。具体设计如表 4.21 所示。

表 4.21 pve_networktopology 表

序号	名称	类型	不为空	注释
1	id	bigint	√	拓扑 ID,自增主键
2	name	varchar (150)	√	拓扑名称
3	description	longtext	√	拓扑描述
4	is_active	tinyint (1)	√	是否启用
5	diagram_data	json	√	拓扑图数据 (Cytoscape.js 元素 JSON)
6	metadata	json	√	拓扑元信息 (布局、样式等)
7	created_at	datetime	√	创建时间
8	updated_at	datetime	√	最后修改时间

chat_aiapikey 表 (AI API Key 表)

AI API Key 表集中管理各大语言模型服务商的 API 密钥,Key 值以加密形式存储,并记录调用统计与错误信息,支持按优先级进行 Key 轮换。具体设计如表 4.24 所示。

表 4.24 chat_aiapikey 表

序号	名称	类型	不为空	注释
1	id	bigint	√	Key ID,自增主键
2	provider	varchar (50)	√	服务提供商 (openai/deepseek 等)
3	key_name	varchar (100)	√	Key 名称标识
4	api_key_encrypted	longtext	√	API Key 密文 (加密存储)

5 base_url varchar (255) 自定义 API 基础地址
 6 is_active tinyint (1) √ 是否启用
 7 priority int √ 使用优先级 (数值越小优先级越高)
 8 daily_token_limit bigint 每日 Token 用量限额
 9 monthly_token_limit bigint 每月 Token 用量限额
 10 daily_tokens_used bigint √ 今日已用 Token 数
 11 monthly_tokens_used bigint √ 本月已用 Token 数
 12 total_tokens_used bigint √ 累计已用 Token 数
 13 total_cost decimal (10,2) √ 累计费用估算
 14 last_used_at datetime 最近使用时间
 15 error_count int √ 累计错误次数
 16 created_at datetime √ 创建时间

chat_aiuserquota 表 (用户 AI 配额表)

用户 AI 配额表为每位用户单独配置 Token 用量限额,系统在每次模型调用后更新已用量,超额后拒绝新请求,支持按日/月维度的配额类型。具体设计如表 4.25 所示。

表 4.25 chat_aiuserquota 表

序号	名称	类型	不为空	注释
1	id	bigint	√	记录 ID,自增主键
2	user_id	int	√	关联用户 (外键 → auth_user)
3	quota_type	varchar (20)	√	配额类型 (daily/monthly/total)
4	token_limit	bigint	√	Token 限额上限
5	tokens_used	bigint	√	已用 Token 数
6	reset_at	datetime		下次重置时间
7	is_active	tinyint (1)	√	是否启用该配额
8	created_at	datetime	√	创建时间

chat_aiagentconfig 表 (AI 智能体配置表)

AI 智能体配置表存储各实验专属 AI Agent 的配置信息,通过 bounded_experiment_id 与实验绑定,支持 RAG 检索增强与对话记忆功能的独立开关。具体设计如表 4.26 所示。

表 4.26 chat_aiagentconfig 表

序号	名称	类型	不为空	注释
1	id	bigint	√	智能体 ID,自增主键
2	agent_key	varchar (50)	√	智能体唯一标识键 (唯一)
3	agent_name	varchar (100)	√	智能体名称
4	model_config_id	bigint		关联模型配置 (外键 → chat_aimodelconfig)
5	bounded_experiment_id	bigint		绑定实验 (外键 → experiments_courseexperiment)
6	system_prompt	longtext	√	系统提示词 (System Prompt)
7	context_type	varchar (50)	√	上下文类型
8	temperature	decimal (3,2)	√	温度参数
9	max_tokens	int	√	最大 Token 数
10	enable_rag	tinyint (1)	√	是否启用 RAG 检索增强
11	rag_top_k	int	√	RAG 检索返回文档块数量
12	enable_memory	tinyint (1)	√	是否启用对话记忆
13	memory_window	int	√	记忆窗口大小 (保留最近 N 轮)
14	language	varchar (10)	√	回复语言 (zh/en)
15	is_active	tinyint (1)	√	是否启用
16	created_at	datetime	√	创建时间

chat_chatconversation 表（对话会话表）

对话会话表记录每次用户与 AI 助手之间的完整对话会话,通过 session_id 唯一标识,支持上下文类型区分（通用对话/实验专项对话）,记录会话消息数与最后活跃时间。具体设计如表 4.27 所示。

表 4.27 chat_chatconversation 表

序号	名称	类型	不为空	注释
1	id	bigint	√	会话 ID,自增主键
2	user_id	int	√	发起用户（外键 → auth_user）
3	session_id	varchar (64)	√	会话唯一标识（UUID,唯一）
4	title	varchar (255)	√	会话标题
5	context_type	varchar (50)	√	上下文类型（general/experiment）
6	context_id	varchar (100)	√	上下文对象 ID（如实验 ID）
7	model_name	varchar (50)	√	使用的模型名称
8	temperature	decimal (3,2)		对话温度参数
9	max_tokens	int		最大 Token 限制
10	message_count	int	√	累计消息条数
11	last_message_at	datetime		最后消息时间
12	is_archived	tinyint (1)	√	是否已归档
13	created_at	datetime	√	会话创建时间

chat_chatmessage 表（对话消息表）

对话消息表存储每条对话消息的详细内容与 Token 消耗信息,通过 sequence 字段保证消息的顺序性,支持多轮对话的消息追溯与 Token 统计。具体设计如表 4.28 所示。

表 4.28 chat_chatmessage 表

序号	名称	类型	不为空	注释
1	id	bigint	√	消息 ID,自增主键
2	conversation_id	bigint	√	所属会话（外键 → chat_chatconversation）
3	role	varchar (20)	√	消息角色（user/assistant/system）
4	content	longtext	√	消息正文
5	content_type	varchar (20)	√	内容类型（text/markdown）
6	sequence	int	√	消息在会话中的顺序编号
7	prompt_tokens	int	√	输入 Token 消耗
8	completion_tokens	int	√	输出 Token 消耗
9	total_tokens	int	√	总 Token 消耗
10	model_name	varchar (50)	√	实际使用的模型名称
11	finish_reason	varchar (50)	√	生成结束原因（stop/length 等）
12	feedback	varchar (20)	√	用户反馈（like/dislike/none）
13	error_message	longtext	√	错误信息（调用失败时记录）
14	retry_count	int	√	重试次数
15	parent_message_id	bigint		父消息 ID（外键,用于分支对话）
16	created_at	datetime	√	消息创建时间

4.3.2 E-R图设计

根据以上各实体的关联关系,系统核心业务数据的 E-R 图如图 4.3 到图4.xxx所示。图中展示了用户权限管理（User-Role-Menu/Permission）、实验教学（Experiment-Guidebook-Submission）与 AI 助手（ModelConfig-ApiKey-Conversation-Message）三条主要业务关联链路,以及 PVE 资源（PveServer-VirtualMachine/LxcContainer）的独立管理链路。

由于系统整体业务数据的 E-R 图较为复杂,因此将整个E-R图拆分成了用户权限管理模块（涉及数据表有xxxx）、实验教学模块（涉及表数据表有xxxxxx）具体内容详见图4.3到图4.xxx。

用户权限管理模块管理模块如图4.3所示,用户表(表auth_user)实体与用户组(表auth_group)的关系是多对一、通过关系表auth_user_groups记录。xxxx Comment by mecio: 每个E-R图的内容需要用一段文字简单描述。

如果图太多了,可以减少1-2个。

Comment by mecio: 图存在的问题是:

不应该出现图1.xxxxz这种字眼,图标题统一放在图的下方。

有多个图,不能放在一起,要分开说明。

图4.3 用户认证与权限管理模块E-R图。

图4.4 RBAC角色与组织架构模块E-R图。

4.4 本章小结

本章从系统设计原则、系统总体设计与数据库设计三个层面,对基于 PVE 的虚拟化云管平台进行了全面的工程方案设计。

Comment by mecio: 缩进对齐

在系统设计原则方面,确立了前后端分离、模块化设计、安全性优先、接口标准化与用户体验优先五项核心原则,为后续开发提供了明确的技术指导方向。

在系统总体设计方面,提出了前端交互展现层、后端业务代理层与底层基础设施层的三层架构体系;设计了仪表盘、系统管理、PVE 资源管理、实验教学与 AI 助手五大功能模块,并对各模块的功能边界与关键交互逻辑进行了详细说明。

在数据库设计方面,梳理了系统全部 28 张业务数据表,对全部表逐一进行了数据字典设计(表 4.2 至表 4.29),涵盖用户权限(auth_user/rbac_*系列)、PVE 资源(pve_*系列)、实验教学(experiments_*系列)与 AI 助手(chat_*系列)四类核心实体的完整字段定义,为第五章的系统实现提供了数据基础与接口规范依据。

5.系统实现

5.1 系统登录与权限管理模块的实现

5.1.1 用户登录与 JWT 令牌认证的实现

系统登录页面基于 Vben Admin 框架构建,页面中央呈现登录卡片,包含用户名输入框、密码输入框及登录按钮,整体风格简洁统一,符合管理后台的视觉规范。用户填写凭据后,前端对输入内容进行非空与格式校验,校验通过后通过 Axios 以 POST 方式将用户名与密码提交至后端接口 /api/v1/auth/login/。登录成功后,系统在页面顶部展示欢迎提示,并自动跳转至仪表盘主界面;若认证失败,则在表单下方显示相应的错误提示信息。页面实现如图5.1所示。

后端登录视图在接收到请求后,首先从请求体中提取用户名与密码字段,调用 Django 内置的 authenticate () 函数完成身份验证,其内部通过 check_password () 对数据库中存储的加盐哈希密码进行比对。认证通过后,视图使用 SimpleJWT 库的 RefreshToken.for_user () 方法为该用户生成一对令牌:AccessToken (默认有效期较短,约为1小时) 与 RefreshToken (有效期较长,约为7天)。生成的两枚令牌随 HTTP 200 响应一并返回给前端。前端在 Pinia store 中将 AccessToken 与 RefreshToken 持久化存储,并在 Axios 请求拦截器中为每次后续请求的 Header 自动附加 Authorization: Bearer <accessToken>。当 Axios 响应拦截器捕获到 HTTP 401 状态码时,系统会自动携带 RefreshToken 向 /api/v1/auth/token/refresh/ 发起静默刷新请求,刷新成功后无缝重放原始请求,从而实现用户无感知的 Token 续期机制。相关代码如图5.2所示。 Comment by mecio: 图片代码太多,要缩减后再截图,只留关键代码。

背景不能是黑色,要白色或者浅灰色都行

5.1.2 RBAC 权限控制与动态菜单下发的实现

权限管理页面提供角色列表、菜单树配置与权限规则三个子模块的管理界面。角色管理以表格形式呈现所有角色,支持新增、编辑与删除操作;菜单树配置页以层级树形结构展示全部菜单节点,管理员可通过复选框为角色勾选可见菜单;权限规则页展示各角色对应的 API 访问规则,包含请求方法与 URL 匹配模式。页面实现如图5.3所示。

【图5.3 RBAC 权限管理页面】 Comment by mecio: 不能将几张图放在一起作为一个图,这种大的建议每个是一个单独的图。

用户登录完成后,前端随即调用 /api/v1/rbac/menus/ 接口获取当前用户的菜单树。后端视图通过 request.user 关联查询 rbac_userrole 表,获取用户所属的所有角色,再经 rbac_role → rbac_role_menus → rbac_menu 的关联链路,汇总出该用户有权访问的全部菜单节点,以树形结构序列化后返回。前端收到菜单数据后,Vben Admin 的全局路由守卫 beforeEach 钩子依据菜单列表动态注册路由,未包含在返回菜单中的路由路径即使被手动输入也无法激活对应页面组件,从而在前端层面形成路由隔离。在后端层面,系统实现了自定义 API 权限中间件,每个请求到达时,中间件遍历请求用户所关联

角色的全部 `rbac_role_permissions` 记录,将当前请求的 HTTP Method 与 Path 与权限规则中的 `url_pattern` 进行正则匹配,若未命中任何规则则直接返回 HTTP 403 响应,杜绝越权调用的可能。相关代码如图5.4所示。

【图5.4 动态菜单下发与权限中间件代码实现】

5.1.3 用户管理与组织架构管理的实现

用户管理页面以数据表格为主体,展示系统内所有用户的用户名、真实姓名、邮箱、所属组织、角色标签及账号状态等字段,表格顶部提供关键词搜索框与状态下拉过滤器,支持组合条件筛选。点击"新增"按钮可弹出用户创建表单,填写基本信息并选择组织与角色后提交;点击操作列中的"编辑"按钮可修改用户信息,点击"停用"可禁用该账号登录。组织架构管理页以树形组件展示院系与班级的层级关系,各节点支持拖拽调整排列顺序,并可通过右键菜单或操作按钮进行新增子节点、编辑名称与删除等操作。页面实现如图5.5所示。

——【图5.5 用户管理与组织架构管理页面】

后端 `UserViewSet` 继承自 DRF 的 `ModelViewSet`,在处理用户创建请求时,视图会从请求数据中提取明文密码字段,调用 Django 的 `make_password()` 函数对其进行加盐哈希处理后再写入数据库,确保密码不以明文形式落盘。在更新用户角色时,视图首先删除 `rbac_userrole` 表中该用户的全部旧角色关联记录,随后将前端传入的新角色 ID 列表批量创建为新的关联记录,以此保证角色绑定关系的一致性。组织架构的树形数据由后端递归序列化生成,并支持通过 `parent_id` 字段表达父子层级关系,前端在提交拖拽排序结果时携带 `sort_order` 字段,后端批量更新各节点的排序权重。相关代码如图5.6所示。

——【图5.6 用户管理后台逻辑代码实现】

5.2 仪表盘模块的实现

仪表盘页面是用户登录后首先看到的总览界面,整体采用栅格布局,分为顶部统计区、中部图表区两大区域。顶部排列四张统计卡片,依次显示当前接入的 PVE 节点数量、系统累计消耗的 AI Token 数、已创建实验总数及注册用户总数,每张卡片附有对应的矢量图标与趋势指示。中部左侧为 ECharts 渲染的双轴组合图表,以柱状图展示近14天每日 AI Token 消耗量(对应左 Y 轴),以折线图展示同期调用次数(对应右 Y 轴),二者共享同一 X 轴时间轴;右侧并排展示三个圆形进度条,分别代表当前 PVE 节点的 CPU 使用率、内存使用率与磁盘使用率,进度条颜色随使用率阈值动态变化,低负载显示绿色、中等负载显示橙色、高负载显示红色,直观反映资源压力状况。页面实现如图5.7所示。

——【图5.7 仪表盘页面】

前端在页面挂载时通过 `Promise.allSettled` 并发起四个统计接口请求,该方式确保任意单个接口失败不会影响其余卡片的数据展示。节点资源使用率数据来自接口 `/api/v1/pve/nodes/{server_id}/{node_name}/monitor/?timeframe=hour`,后端接收请求后,以反向代理的方式向目标 PVE 服务器的 `/api2/json/nodes/{node}/rrddata` 接口发起请求,解析返回的 RRD 时序数据中的 `cpu`、`mem`、`disk` 指标,取最新一条记录计算实际使用率后返回给前端。ECharts 图表在初始化时传入双 Y 轴配置对象,柱状图系列与折线图系列分别绑定至左侧与右侧 Y 轴刻度,并在 tooltip 中统一格式化显示两项指标的数值,方便管理员对比分析。相关代码如图5.8所示。

——【图5.8 仪表盘数据聚合与图表渲染代码实现】

5.3 PVE 资源管理模块的实现

5.3.1 PVE 服务器管理与虚拟机列表的实现

PVE 服务器管理页面以表格形式列出已接入平台的所有 PVE 服务器,展示服务器名称、主机地址、端口号及当前连接状态等信息。每行末尾的操作列提供"编辑"与"连通性测试"按钮,点击"连通性测试"后按钮进入 loading 状态,待后端验证完毕后在状态列更新结果图标。添加新服务器时弹出抽屉表单,填写主机、端口、认证凭据等信息后提交保存。虚拟机列表页展示当前所有已同步的 KVM 虚拟机记录,字段包括 VMID、所属服务器、所属节点、虚拟机名称、运行状态(以 Tag 组件着色区分 running/stopped)、CPU 核数、内存容量与磁盘大小;列表顶部提供"同步 PVE"按钮,点击后触发全量同步任务。页面实现如图5.9所示。

——【图5.9 PVE 服务器管理与虚拟机列表页面】

连通性测试接口接收服务器 ID 后,后端取出该记录的主机与端口信息,使用系统内置的 PVE Session 向目标服务器的 `/api2/json/nodes` 端点发起 GET 请求,若响应状态码为 200 则判定连通正常并更新数据库中的 `status` 字段为 `online`,否则将错误原因写入响应体并将状态置为 `offline`,返回给前端展示。虚拟机同步逻辑由后端视图遍历数据库中所有处于激活状态的 `PveServer` 记录,逐一向其每个节点的 `/api2/json/nodes/{node}/qemu` 接口发起请求,将返回的虚拟机列表与数据库 `pve_virtualmachine` 表进行比对:若 VMID 不存在则创建新记录,若已存在则更新名称、状态、CPU、内存等字段,最终返回包含 `created` 与 `updated` 数量的汇总结果。相关代码如图5.10所示。

— 【图5.10 PVE 服务器管理与虚拟机同步代码实现】

5.3.2 虚拟机全生命周期管理与快照备份的实现

虚拟机详情页采用左侧竖向 Tab 导航布局,标签项包括概要、控制台、硬件、选项、备份与快照六个功能区。概要 Tab 以信息卡片形式展示虚拟机的实时运行状态 (CPU 占用、内存占用、运行时长) 与基本配置信息 (操作系统类型、磁盘大小、网卡 MAC 地址等)。快照 Tab 以树形列表展示该虚拟机的所有快照节点及创建时间,支持在列表中发起新建快照与回滚操作。备份 Tab 展示历史备份任务记录,支持触发手动备份。页面顶部操作区提供电源控制下拉菜单 (启动、关机、强制停止、重启),并在删除操作时弹出二次确认对话框,要求用户手动输入目标 VMID 方可确认删除,防止误操作。页面实现如图5.11所示。

— 【图5.11 虚拟机全生命周期管理页面】

虚拟机创建流程采用向导式多步表单,用户依次填写 CPU 核数、内存大小 (MB)、磁盘大小 (GB)、存储池名称与模板 VMID,提交后前端 POST 至 `/api/v1/pve/vms/`。后端视图接收参数后,向 PVE 发起 `POST /api2/json/nodes/{node}/qemu/{template_vm}/clone` 请求,PVE 立即返回任务追踪码 UPID,后端将 UPID 与新虚拟机记录存入数据库,并由 Celery 异步 Worker 以轮询方式持续查询 `GET /api2/json/nodes/{node}/tasks/{upid}/status` 直至任务完成,随后更新虚拟机状态字段。电源操作视图接收操作类型参数 (start/shutdown/stop/reboot),映射至对应的 PVE 接口路径 `/api2/json/nodes/{node}/qemu/{vmid}/status/{action}` 并以 POST 方式调用。快照管理通过 GET 请求 `/api2/json/nodes/{node}/qemu/{vmid}/snapshot` 获取快照列表,通过 POST 创建新快照,回滚操作则调用 `POST /api2/json/nodes/{node}/qemu/{vmid}/snapshot/{name}/rollback`,所有操作均由后端代理转发并统一处理异常响应。相关代码如图5.12所示。

— 0

【图5.12 虚拟机生命周期管理代码实现】

5.3.3 虚拟机 noVNC 图形控制台的实现

虚拟机图形控制台嵌入在虚拟机详情页的控制台 Tab 中,页面初始状态显示连接提示与“打开控制台”按钮,点击后进入连接流程,连接成功后控制台区域渲染出虚拟机的图形桌面画面。工具栏固定于控制台区域顶部,提供全屏切换、发送 `Ctrl+Alt+Del` 组合键、电源操作 (关机/重启) 等快捷按钮,方便用户在不退出控制台的情况下进行基本管理操作。页面实现如图5.13所示。

— 【图5.13 虚拟机 noVNC 控制台页面】

控制台连接流程分为两个阶段。第一阶段,前端激活控制台 Tab 时,向 `/api/v1/pve/vms/{id}/console/ticket/` 发起 POST 请求,后端生成一枚有效期为60秒的一次性 Ticket 并返回,该 Ticket 存储于 Django Cache中,与虚拟机 ID 和用户 ID 绑定。第二阶段,前端以 `?ticket=xxx` 为 URL 参数,发起至 `wss://.../ws/console/vm/{vm_id}/` 的 WebSocket 连接。后端 VmConsoleConsumer 在 WebSocket 握手阶段从 URL 参数中提取 Ticket,与 Cache 中的记录进行校验,验证通过后立即将该 Ticket 作废以防重放攻击;随后向 PVE 申请 VNC Ticket (`POST /api2/json/nodes/{node}/qemu/{vmid}/vncproxy`),利用 PVE 返回的 WebSocket URL 与端口,建立后端至 PVE noVNC WebSocket 的持久连接,并将来自前端的数据帧与来自 PVE 的数据帧进行双向透明转发。前端使用 `@novnc/novnc` 库的 RFB 类,将后端 WebSocket 作为传输层,最终在页面 canvas 元素上渲染出虚拟机实时图形桌面。相关代码如图5.14所示。

【图5.14 noVNC 控制台 WebSocket 隧道代码实现】

5.3.4 LXC 容器管理与 WebSSH 终端的实现

LXC 容器管理页面与虚拟机管理页面采用一致的布局结构,表格字段包括容器 ID、所属服务器、所属节点、容器名称、运行状态、CPU 与内存配置等。详情页同样采用竖向 Tab 导航,差异在于控制台 Tab 使用基于 `xterm.js` 的命令行终端界面而非 noVNC 图形界面。`xterm.js` 终端区域占据控制台 Tab 的主体空间,工具栏提供主题切换 (VS Code Dark、Monokai 等预设主题)、字体大小调节与清屏按钮。页面实现如图5.15所示。

— 【图5.15 LXC 容器管理与 WebSSH 终端页面】

WebSSH 的实现机制借鉴了 noVNC 的设计思路。系统首先通过 REST 接口获取单次有效的票据 (Ticket),随后与 wss 协议下的控制台路径建立 WebSocket 通信。后端 LxcConsoleConsumer 在完成凭证校验后,利用 Paramiko 或 `asyncssh` 异步库建立指向目标容器 IP 的 SSH 隧道,并在认证通过后挂载伪终端 (PTY) 以开启交互式会

话。

数据的双向传递由两组并行协程驱动:其一负责将 WebSocket 捕获的指令实时注入 SSH 通道的标准输入 (stdin);其二则动态监测通道输出 (stdout/stderr),并将回显字节流推送至前端。在视图层,xterm.js 配合 FitAddon 与 WebLinksAddon 插件实现了终端布局自适应及 URL 自动解析;通过监听 onmessage 消息序列执行 terminal.write () 渲染画面,同时将用户的键盘操作同步回馈至后端,从而构建了低延迟的 Web 端 shell 交互环境(代码逻辑详见图 5.16)

WebSSH 实现流程与 noVNC 类似,同样先通过 REST 接口申请一次性 Ticket,再建立至 wss://.../ws/console/lxc/{lxc_id}/ 的 WebSocket 长连接。后端 LxcConsoleConsumer 完成 Ticket 验证后,使用 Paramiko (asyncssh) 库向容器对应的 IP 地址发起 SSH 连接,认证成功后开启一个交互式 Shell session,并为其分配伪终端 (PTY)。消费者在两条异步协程中分别处理输入与输出:收到来自前端 WebSocket 的键盘输入字节时,直接写入 SSH channel 的 stdin;SSH channel 的 stdout 与 stderr 产生新数据时,立即通过 WebSocket 推送至前端。前端 xterm.js 配置了 FitAddon 以根据容器尺寸自动调整终端行列数,配置了 WebLinksAddon 以识别并高亮输出中的 URL 链接;通过 WebSocket 的 onmessage 事件将接收到的字节流调用 terminal.write () 写入终端画布,用户在 xterm.js 中的每次击键事件则通过 WebSocket send () 实时发往后端,实现近乎零延迟的 Web Shell 交互体验。相关代码如图 5.16 所示。

【图 5.16 WebSSH 终端 WebSocket 代码实现】

5.3.5 网络拓扑可视化的实现

网络拓扑页面包含"实时拓扑"与"拓扑设计"两个切换 Tab。实时拓扑 Tab 展示从 PVE 节点动态获取的当前网络结构,使用 Cytoscape.js 以节点与连线的形式可视化呈现物理网卡、网桥及 VLAN 之间的连接关系;节点以不同图标区分类别(物理网卡、虚拟网桥),点击任意节点在右侧信息面板中显示该网络接口的 IP 地址、子网掩码、绑定状态等详情;顶部提供"导入到设计器"按钮,可将当前实时拓扑数据一键迁移至拓扑设计 Tab 中进行二次编辑。拓扑设计 Tab 提供交互式画布,左侧工具栏包含添加节点、添加连线、删除元素等操作按钮,用户可自由拖拽节点位置、双击编辑标签,并在完成设计后点击"保存"提交。页面实现如图 5.17 所示。

【图 5.17 网络拓扑可视化页面】

实时拓扑数据由后端接口提供,视图接收节点名称参数后,向 PVE 的 /api2/json/nodes/{node}/network 接口发起请求,将返回的网卡与网桥配置数据解析并转换为 Cytoscape.js 所需的元素格式(包含 nodes 数组与 edges 数组),其中每个 node 对象携带接口类型、IP 等数据字段,edge 对象描述网桥与物理网卡的绑定关系。前端接收到数据后,以 layout: { name: 'cose' } 弹力布局算法自动排列节点,避免元素重叠。拓扑设计的保存操作由前端调用 cy.json () 将当前画布的完整状态序列化为 JSON 对象,POST 至 /api/v1/pve/topologies/,后端将其存入 pve_networktopology 表的 diagram_data JSON 字段。拓扑管理列表页支持通过 URL 参数 ?id=X 加载指定已保存的拓扑方案,前端在页面挂载时检测该参数,若存在则调用对应 GET 接口取回 diagram_data,再通过 cy.json () 的逆操作恢复画布状态。相关代码如图 5.18 所示。

【图 5.18 网络拓扑数据解析与 Cytoscape.js 渲染代码实现】

5.4 实验教学模块的实现

5.4.1 教师端实验管理的实现

教师端实验列表页采用卡片式布局,每张卡片展示实验标题、状态 Tag(草稿为灰色、已发布为绿色、已归档为蓝色)、课程代码、难度等级、开放与截止时间区间,以及提交率、批改率与总分上限等统计数字;顶部搜索区提供标题关键词输入框与状态下拉过滤器,支持组合筛选。实验创建与编辑页采用步骤表单布局,依次填写实验基本信息(标题、课程代码、描述、开放/截止时间)与评分配置(评分项名称、分值,以 JSON 格式管理);指导书上传区支持拖拽上传 PDF 或 Word 格式文件,上传完成后卡片列表显示上传进度。提交管理页以表格形式展示所有学生的提交记录,字段包括学生姓名、提交时间、是否逾期、批改状态与得分,支持按实验与批改状态过滤;批改页左侧渲染学生提交的 Markdown 报告内容,右侧提供评分表单(含总分输入框、各评分项分值输入框及评语文本区),操作按钮包括"提交评分"与"退回并附原因"。页面实现如图 5.19 所示。

【图 5.19 教师端实验管理页面】

实验创建视图在保存指导书文件时,将文件记录存入 experiments_courseguidebook 表,并在事务提交后异步触发 Celery 任务执行 RAG 向量化索引构建。实验发布操作通过 PATCH 请求访问 /api/v1/experiments/{id}/publish/,后端自定义 Action 视图将该实验的 status 字段更新为 published,同时校验发布前置条件(如截止时间必须晚于当前时

间)。批改视图在接收到评分数据后,将各项分值汇总计算总分写入 experiments_coursesubmission 的 score 字段,并将 submission_status 更新为 graded。退回操作将状态置为 returned 并写入退回原因,学生可据此修改后重新提交。成绩导出接口 GET /api/v1/experiments/{id}/export_grades/ 由后端使用 openpyxl 库动态生成 Excel 文件,工作表包含学号、姓名、得分、是否逾期等字段,以 application/vnd.openxmlformats-officedocument.spreadsheetml.sheet 内容类型返回文件流,前端触发浏览器文件下载。相关代码如图5.20所示。

【图5.20 教师端实验管理后台代码实现】

5.4.2 学生端实验工作台的实现

学生端实验列表页屏蔽所有教师管理功能入口,仅展示当前用户有权访问的已发布且处于有效期内的实验卡片,卡片信息精简为标题、截止时间倒计时与当前提交状态。点击卡片进入报告编辑器页,页面采用左右分栏 Flex 布局:左侧面板展示指导书预览(PDF 内嵌或 HTML 渲染),右侧面板为 Markdown 编辑器区域,编辑器下方工具栏提供保存草稿、提交报告按钮,顶部状态栏显示当前提交状态与最后保存时间。成绩查看页在批改完成后展示各评分项得分明细、总分与教师评语;若报告被退回,则在页面顶部显示退回原因提示框,并激活重新编辑按钮。页面实现如图5.21所示。

【图5.21 学生端实验工作台页面】

学生首次进入实验编辑器时,后端视图自动为其创建一条状态为 draft 的 experiments_coursesubmission 记录(通过 get_or_create 实现幂等性),返回 submission ID 供后续操作使用。Markdown 编辑器基于 marked.js 实现实时预览,支持图片拖拽上传:用户将图片文件拖入编辑区域时,前端捕获 dragover 与 drop 事件,将图片文件以 multipart/form-data 格式 POST 至 /api/v1/experiments/submissions/{id}/attachments/,后端将文件保存至服务器并写入 experiments_courseattachment 表,返回文件访问 URL,前端随即在光标位置插入 Markdown 图片语法。保存草稿操作通过 PATCH 请求将编辑器内容写入 submission.content 字段。提交操作在二次确认弹框中向 PATCH 接口发送 submission_status=submitted,后端记录 submit_time 并与 experiment.end_time 比较,若超时则置 is_late=True,确保逾期判断的准确性。相关代码如图5.22所示。

【图5.22 学生端实验工作台代码实现】

5.5 AI 助手模块的实现

5.5.1 模型配置的实现

模型管理页面以表格形式展示平台内所有已配置的大语言模型条目,字段包括模型标识键(用于代码中调用的唯一键名)、显示名称、模型类型(Chat/Embedding)、关联的 API Key 名称、温度参数与启用状态开关;支持通过"新增"按钮添加新模型配置,通过编辑操作修改温度、最大 Token 数等参数。API Key 管理页展示已添加的所有密钥,密钥值经过脱敏处理,仅显示前4位字符与后4位字符,中间以星号代替,防止密钥泄露。额度管理页以表格形式展示每位用户的 Token 配额信息,字段包括用户名、配额类型、Token 上限、已使用量与下次重置时间,管理员可直接在表格行内点击编辑修改某用户的配额上限。页面实现如图5.23所示。

【图5.23 模型配置】

后端在保存 API Key 时,不将原始密钥字符串直接存入数据库,而是使用 Django 的 cryptography 库(Fernet 对称加密方案)对密钥值进行加密,将密文字符串存入 ai_apikey.api_key_encrypted 字段;在实际调用大模型 API 时,后端从数据库读取密文并实时解密还原为明文传递给 openai SDK,从而避免密钥以明文形式暴露在数据库中。额度查询与扣减通过数据库事务与行级锁(select_for_update())确保并发安全,防止多个并发请求同时读取相同余额导致超额消费。相关代码如图5.24所示。

【图5.24 模型管理代码实现】

5.5.2 RAG 知识库的实现

知识库管理页面以列表形式展示每份已上传指导书的向量化索引状态(pending/processing/done/failed)、文档分块数量与最后更新时间,若状态为 failed 则在行内展示错误摘要;每行末尾提供"手动重建"按钮以重新触导向量化任务。智能体管理页展示系统内所有已配置的 AI 智能体条目,字段包括 agent_key、绑定的模型、关联实验、系统提示词摘要、RAG 开关状态与检索 Top-K 值,点击编辑可修改完整的 System Prompt 文本与检索参数。页面实现如图5.25所示。

【图5.25 RAG 知识库配置页面】

RAG 向量化流程由 Celery Worker 异步执行:任务启动后首先将 chat_aiknowledgeindexstatus 表中对应记录的状态更新为 processing,随后读取指导书文件内容,按 chunk_size=500、overlap=50 的滑动窗口策略将文本切分为多个片段,逐批调用 Embedding 模型接口生成向量,最终将文档块文本与对应向量写入 ChromaDB 向量数据库的专

属 Collection 中,并将状态更新为 done,记录分块总数;若执行过程中发生异常则将状态置为 failed 并记录异常信息。智能体在处理用户问题时,若 enable_rag=True,后端将用户问题文本同样转换为向量,在 ChromaDB 中执行相似度检索,取 Top-K 个最相关文档块,将其内容拼接至 System Prompt 上下文后再提交给大模型,从而使回答更贴合平台实际文档内容,有效降低模型幻觉。相关代码如图5.26所示。

图5.26 RAG 向量化与智能体检索代码实现

5.5.3 流式对话 WebSocket 实现

AI 助手对话窗口以浮动抽屉组件的形式嵌入平台所有页面,点击右上角悬浮图标即可展开对话界面。对话区域上方为消息气泡滚动列表,区分用户消息(右侧蓝色气泡)与 AI 回复(左侧白色气泡),AI 回复气泡下方提供点赞与踩反馈按钮;底部为消息输入框,支持多行文本输入,回车键或点击发送按钮提交消息。AI 回复内容在流式接收过程中以打字机效果逐字追加渲染,并支持 Markdown 格式解析(含代码块高亮显示),提升阅读体验。页面实现如图5.27所示。

图5.27 AI 助手流式对话页面

前端在对话窗口打开时,先通过 REST 接口申请一枚一次性鉴权 Ticket,再以该 Ticket 作为 URL 参数建立至 wss://.../ws/ai/chat/ 的 WebSocket 长连接,此后该连接在对话会话期间保持打开状态以复用。后端 AIConsumer 在收到用户消息帧后,按照以下流程处理:首先查询 chat_aiuserquota 表验证用户剩余 Token 配额,若配额不足则通过 WebSocket 返回错误提示并终止处理;若 enable_rag 为真,则将用户问题向量化后在 ChromaDB 中检索相关文档块,拼入上下文;随后使用 openai SDK 以 stream=True 参数调用大模型 API,在异步 for 循环中逐个读取响应 chunk,每读取到一个 token 片段即立即通过 WebSocket 将其推送至前端,实现逐字流式输出;待流式响应完成后,统计本次对话消耗的总 Token 数,将完整对话记录写入 chat_chatmessage 表,同时在 chat_aisagelog 中记录用量明细,并对 chat_aiuserquota 中的 tokens_used 字段执行原子递增扣减。相关代码如图5.28所示。

图5.28 AI 助手流式对话 WebSocket 消费者代码实现

6.系统测试

6.1 测试目的

本章对基于 Vue 3 与 Django 5 的 PVE 虚拟化管理系统进行系统性功能测试,旨在验证系统各模块的功能完整性与正确性,确认系统实现满足第三章需求分析中提出的各项功能性需求。测试覆盖系统登录与权限管理、仪表盘数据展示、PVE 资源管理、实验教学、AI 助手以及系统管理与审计共六大核心模块,通过设计典型测试用例,逐项验证系统在正常输入条件下的功能表现,保证系统在交付前达到预期的设计目标与质量标准。

6.2 测试环境

硬件环境方面,测试部署于 Proxmox VE 物理服务器上创建的一个 LXC 容器实例,该容器配置为 4 核 CPU、8 GB 内存与 100 GB 磁盘,运行 ubuntu 操作系统,为系统后端服务、数据库及前端静态文件提供统一的运行环境。

软件环境方面,后端采用 Python 3.12 运行,Web 框架为 Django 5.0,配合 Django REST Framework 提供 RESTful API 服务,Django Channels 与 Daphne ASGI 服务器支持 WebSocket 实时通信;前端基于 Vue 3 与 Vben Admin 框架构建,由 Nginx 进行静态文件托管;数据存储层使用 MySQL 8.0 作为关系型数据库;测试所用客户端为 Chrome 浏览器。

网络环境方面,通过 frp (Fast Reverse Proxy) 内网穿透工具,将 LXC 容器内的各服务端口映射至公网服务器的指定端口,可通过公网域名或IP直接访问系统前端及后端接口,模拟真实用户的使用场景。

6.3 功能测试

6.3.1 系统登录与权限管理模块测试

系统登录与权限管理模块是整个系统的安全基础,负责用户身份认证、访问令牌管理及基于角色的访问控制 (RBAC)。本节测试覆盖用户登录的正常与异常流程、JWT 令牌的签发与自动刷新机制、不同角色用户登录后的动态菜单下发逻辑、前端路由守卫对越权访问的拦截效果,以及后端 API 接口对低权限用户的访问控制能力。此外,还对管理员执行用户账号管理及组织架构管理等基础运维操作进行了验证,确保权限体系在各层面均能有效运作。

表6.1 系统登录与权限管理测试用例

序号	测试项目	测试内容	结果
1	用户名密码正确登录	输入有效的用户名与密码,点击登录按钮,检查是否返回 JWT AccessToken 与 RefreshToken,并自动跳转至仪表盘页面	通过
2	错误密码登录	输入正确用户名与错误密码,点击登录按钮,检查系统是否拒绝登录并在页面显示"用户名或密码错误"提	

示,且不返回任何令牌 通过

3 AccessToken 过期后自动刷新 等待 AccessToken 过期后,在不重新登录的情况下继续发起 API 请求,检查系统是否自动使用 RefreshToken 完成令牌刷新并无感续期 符合预期

4 动态菜单下发与越权路由拦截 分别使用管理员账号与普通学生账号登录,检查两者侧边栏菜单项是否存在差异;使用学生账号直接访问管理员专属路由,检查是否被重定向或拒绝访问 通过

5 RBAC API 权限拦截 使用低权限用户的 Token 调用需要高权限角色的后端 API 接口,检查接口是否返回 HTTP 403 状态码并拒绝执行操作 通过

6 用户账号管理 以管理员身份进入用户管理页面,依次执行新建用户、编辑用户信息、停用用户账号操作,检查各操作执行后数据是否正确保存并在列表中实时反映 通过

7 组织架构管理 在组织架构管理页面新建院系节点,并在其下创建班级子节点,检查树形结构是否正确展示层级关系,层级节点可正常展开与折叠 符合预期

6.3.2 仪表盘模块测试

仪表盘模块作为系统的数据概览入口,集中展示系统运行状态与关键统计指标。本节测试聚焦于统计卡片数据的加载准确性、ECharts 图表的渲染正确性、PVE 节点资源实时数据的展示效果,以及进度圈颜色阈值逻辑的正确性。同时,针对仪表盘页面同时发起多个数据接口请求的场景,验证在部分接口异常时其余模块能否保持正常展示,确保前端具备良好的容错能力,为用户提供稳定的数据概览体验。

表6.2 仪表盘模块测试用例

序号	测试项目	测试内容	结果
1	统计卡片数据加载	登录后进入仪表盘页面,检查统计卡片区域是否正确显示当前系统的 PVE 节点数、AI Token 消耗总量、实验数量及用户数量,数值与数据库实际数据一致	通过
2	ECharts 双轴图表渲染	查看近 14 日 Token 消耗趋势图,检查柱状图与折线图是否均正确渲染,横轴日期连续,柱状图表示 Token 消耗量,折线图表示调用次数,双轴刻度独立且不重叠	通过
3	PVE 节点资源使用率展示	在仪表盘节点资源区域,检查各 PVE 节点的 CPU 使用率、内存使用率、磁盘使用率是否以进度圈形式实时展示,数值与 PVE 服务器实际状态吻合	符合预期
4	进度圈颜色阈值逻辑	分别观察使用率低于 60%、介于 60%~80% 之间以及超过 80% 的进度圈颜色,检查是否分别呈现绿色、橙色、红色,阈值边界处颜色切换正确	通过
5	多接口并发加载容错	模拟单个数据接口返回错误的场景,检查仪表盘页面中其余统计卡片与图表是否仍能正常加载和展示,不因单一接口失败导致整个页面白屏或崩溃	符合预期

6.3.3 PVE 资源管理模块测试

PVE 资源管控模块作为系统的功能核心,承担了对 Proxmox VE 物理主机及其内部虚拟机、LXC 容器的集中化治理。本节测试旨在验证服务器节点的接入与链路检测、实例元数据的实时同步、电力控制指令的响应度及资源删除保护策略。此外,测试重点还涵盖了 noVNC 远程桌面、WebSSH 交互终端以及动态网络拓扑等核心特性的兼容性,旨在全方位评估系统在基础设施管理层面的可靠性与交互流畅度PVE 资源管理模块是系统的核心功能模块,负责对 Proxmox VE 服务器及其上运行的虚拟机与 LXC 容器进行统一管理与操控。本节测试涵盖 PVE 服务器节点的添加与连通性检测、虚拟机列表同步、电源操作指令下发、安全删除保护机制,以及 noVNC 图形控制台、WebSSH 命令行终端和网络拓扑可视化等高级功能,全面验证系统对 PVE 基础设施的管理能力与交互可靠性。

表6.3 PVE 资源管理模块测试用例

序号	测试项目	测试内容	结果
1	添加 PVE 服务器	在 PVE 节点管理页面填写主机地址、端口号、用户名及密码凭据,点击保存,检查新节点是否成功添加并出现在节点列表中	通过
2	连通性测试	对已添加的在线 PVE 节点执行连通性检测,检查其状态标签是否显示"在线";修改节点地址为无效地址后再次检测,检查状态是否变为"离线"	通过
3	虚拟机列表同步	点击"同步 PVE"按钮,等待同步完成后检查虚拟机列表是否更新,并确认系统返回本次同步新增与更新的虚拟机数量统计信息	符合预期
4	虚拟机电源操作	分别对处于关机状态的虚拟机执行启动、关机、强制停止、重启操作,检查每次操作后虚拟机状态标签是否在界面上实时更新至对应状态	通过

5 虚拟机安全删除 在删除虚拟机的确认对话框中,输入与目标 VMID 不匹配的值,检查确认删除按钮是否保持禁用状态;输入正确 VMID 后,检查按钮是否激活并可正常执行删除 通过

6 noVNC 图形控制台 选中一台运行中的虚拟机,点击打开控制台,检查系统是否成功建立 WebSocket 隧道并在页面中渲染出虚拟机的图形桌面,鼠标键盘输入响应正常 通过

7 LXC 容器 WebSSH 终端 选中一个运行中的 LXC 容器,打开 WebSSH 终端,在终端中输入常用 Shell 命令,检查命令输出是否正确,并测试终端主题切换功能是否生效 符合预期

8 网络拓扑可视化 进入网络拓扑页面,检查实时拓扑图中节点与连线是否正确加载;在拓扑设计器中拖拽节点调整布局后保存,刷新页面后检查布局是否正确还原 通过

6.3.4 实验教学模块测试

实验教学模块支撑系统的核心业务流程,涵盖教师端的实验创建与管理,以及学生端的实验工作台、报告撰写与提交等全流程操作。本节测试按照实验教学的完整业务链路进行设计,依次验证教师创建实验、上传指导书并触发向量化任务、发布实验,以及学生进入工作台、撰写并保存 Markdown 格式报告、提交报告,教师批改录入成绩,最终导出成绩单等各环节的功能正确性,确保模块能够完整支撑实验教学的全流程管理需求。

表6.4 实验教学模块测试用例

序号	测试项目	测试内容	结果
1	教师创建实验	以教师身份进入实验管理页面,填写实验名称、描述及步骤表单,点击保存,检查实验是否以"草稿"状态创建成功,实验卡片在列表中正确展示	通过
2	指导书上传与向量化	在实验详情页上传 PDF 格式的实验指导书,检查文件是否上传成功,并确认系统后台自动触发文档向量化 Celery 任务并进入排队处理状态	符合预期
3	实验发布	对草稿状态的实验点击发布操作,检查实验状态是否变更为"已发布",并确认学生端登录后可在实验列表中看到该实验	通过
4	学生进入实验工作台	以学生身份点击进入已发布实验的工作台,检查系统是否自动为该学生创建草稿提交记录,并正确加载实验指导书内容供学生参阅	通过
5	Markdown 报告编辑与草稿保存	在实验工作台报告编辑器中输入 Markdown 格式的报告内容,点击保存草稿,刷新页面后检查编辑器中内容是否完整还原	通过
6	学生提交报告	在截止时间前点击提交报告,检查提交记录状态是否变更为"已提交",且系统不对该提交标记逾期标志	符合预期
7	教师批改与成绩录入	以教师身份进入批改页面,对已提交的报告录入分数和评语,点击确认批改,检查提交记录状态是否变更为"已批阅",分数和评语正确保存	通过
8	成绩导出 Excel	在成绩管理页面点击导出成绩,检查下载的 Excel 文件是否包含学号、姓名、得分、提交时间等字段,数据与系统记录一致,文件格式可正常被 Excel 软件打开	通过

6.3.5 AI 助手模块测试

AI 助手模块为系统提供大语言模型对话、RAG 知识库检索问答及智能体配置等能力,是系统差异化功能的核心体现。本节测试覆盖大语言模型的配置管理、API Key 的安全脱敏展示、流式对话的前端渲染效果、用户 Token 配额的控制逻辑,以及基于实验指导书向量化数据的 RAG 问答准确性。同时,对为具体实验绑定专属智能体并配置系统提示词 (System Prompt) 的功能进行验证,确保 AI 助手在不同场景下的行为符合预期设计。

表6.5 AI 助手模块测试用例

序号	测试项目	测试内容	结果
1	模型配置	在模型管理页面新增一条大语言模型配置,填写 API 基础地址、模型名称及 API Key,保存后启用该配置,检查配置是否成功保存并处于启用状态	通过
2	API Key 脱敏展示	查看已保存的模型配置详情,检查 API Key 字段是否仅显示首尾各 4 位字符,中间部分以星号遮盖,同时验证该 Key 仍可被系统正常调用以发起模型请求	通过
3	流式对话与 Markdown 渲染	在 AI 助手对话界面发送一条问题消息,检查 AI 回复是否以打字机逐字输出的流式效果呈现,并验证回复中包含的 Markdown 格式内容(如标题、代码块、列表)是否被正确渲染	符合预期
4	Token 配额控制	将某用户的 Token 配额设置为已耗尽状态,以该用户身份发起 AI 对话请求,检查系统是否拒绝调用并返回配额不足的提示信息	通过

5 RAG 知识库问答 确认实验指导书向量化任务已完成,在 AI 助手提出与该指导书内容直接相关的问题,检查系统是否检索到相关文档片段并基于其给出准确回答 符合预期

6 智能体配置与 System Prompt 生效 在智能体管理页面为指定实验绑定专属 Agent,配置特定风格的 System Prompt,进入该实验工作台后与 AI 对话,检查回答风格与 System Prompt 中的指引是否一致 通过

6.3.6 系统管理与审计模块测试

系统管理与审计模块承担着维护系统安全边界与运行透明度的重任。本节测试旨在核实系统是否能够准确、全面地自动生成操作日志,并支持多维度的精细化检索。测试核心还包括对敏感字段持久化过程中的掩码及脱敏效果评估,确保底层数据存储符合安全合规要求。最后,通过对登录轨迹记录及系统级资源指标动态图表的验证,全面评估了系统所具备的异常追溯能力与可视化监控效能系统管理与审计模块承担对系统运行状态的监控及用户操作行为的留存与追溯职责。本节测试重点验证操作日志的自动记录能力及过滤查询功能的准确性,同时检验敏感字段在日志存储时的脱敏处理效果,确保密码、令牌等敏感信息不以明文形式持久化。此外,还对登录日志的完整记录能力以及系统监控页面的进程资源指标展示功能进行测试,以验证系统具备基本的可观测性与安全审计能力。

表6.6 系统管理与审计模块测试用例

序号	测试项目	测试内容	结果
1	操作日志自动记录	以任意用户身份执行新增、修改、删除等写操作（对应 POST、PUT、DELETE 请求）,操作完成后进入操作日志列表,检查是否自动生成对应的日志记录,记录包含请求路径、方法、操作用户及时间等信息	通过
2	操作日志过滤查询	在操作日志查询页面分别按时间范围、用户名、请求方法进行过滤,检查返回的日志列表是否仅包含满足筛选条件的记录,过滤结果准确无误	通过
3	敏感字段脱敏处理	执行涉及密码字段的操作（如修改用户密码）,查看对应操作日志的请求体记录,检查密码字段及 Token 字段是否已被替换为空值或掩码,不以明文形式存储	符合预期
4	登录日志记录	分别执行一次密码正确的登录操作和一次密码错误的登录尝试,进入登录日志列表,检查两次操作是否均产生登录日志记录,记录中包含登录结果（成功/失败）、IP 地址及时间	通过
5	系统监控页面	进入系统监控页面,检查当前 Django 进程的 CPU 使用率、内存占用量及进程运行时长是否实时展示,数值与服务器实际状态吻合	符合预期

6.4 本章小结

本章对 PVE 虚拟化管理系统的六大功能模块进行了系统性功能测试,测试环境基于 LXC 容器部署并通过 frp 内网穿透对外提供访问,贴近真实使用场景。测试内容覆盖系统登录与权限管理、仪表盘数据展示、PVE 资源管理、实验教学、AI 助手以及系统管理与审计模块,共设计并执行了 39 条功能测试用例,全部测试结果均符合预期,无功能缺陷。测试结果表明,系统各模块功能完整,交互逻辑正确,数据处理准确,安全控制有效,系统整体达到了第三章需求分析所规定的各项功能性设计目标,具备正式部署与使用的条件。

7.总结与展望

7.1 论文工作总结

受云计算普及与虚拟化技术迭代的驱动,高校 IT 设施正加速向私有云教学环境转型。尽管 Proxmox VE 提供了稳健的开源虚拟化底座,但在实际教学应用中,其原生面板存在业务抽象不足、权限管控颗粒度较粗等局限。为此,本研究开发了面向教育场景的 PVE 云管理平台,通过重构业务逻辑与权限体系,实现了底层基础设施与高层教学需求的高效对接随着云计算技术的发展与轻量级虚拟化技术的普及,基于私有云平台构建教学与实训环境已成为高校 IT 基础设施建设的重要趋势。Proxmox VE (PVE) 作为一款优秀的开源服务器虚拟化管理平台,提供了强大的底层计算、网络和存储能力。然而,原生 PVE 面板侧重于底层硬件资源调优,缺乏针对教学实验场景的业务逻辑抽象与细粒度权限控制。为此,本文设计并实现了一套基于 PVE 的私有云集群管控与教育实验辅助系统。

在系统的研究与开发过程中,本文主要完成了以下核心工作:

PVE 虚拟化接口的轻量级重构与封装:以 Django 为后端基础,利用 API 代理与 WebSocket 隧道技术,对 PVE 集群底层的虚拟机克隆、启停、网络拓扑生成以及原生 VNC/WebSSH 控制台建立等核心功能进行了 RESTful 规范的二次封装与隔离,为上层业务提供了安全、简便的资源调度接口。

权限体系架构以 RBAC 为核心,通过整合多层级组织实体与分类角色,构建了覆盖 API 路由层级的精细化访问控制屏障。该机制能够完整复现从业务触发至结果响应的每一环节,从而在技术层面保障了平台数据交互过程的合规性及其审计追踪能力细粒度角色权限模型与系统审计机制构建:基于 RBAC（基于角色的访问控制）模型,结合多级组织架构与角色分类,设计了严

格的 API 路由级权限校验防线。同时,通过自定义中间件实现了从发起到响应的全链路操作日志审计机制,确保了平台数据读写合规性与可追溯性。

——沉浸式实训环境与教学闭环管理模块开发:针对教学场景开发了从实验发布、模板自动化克隆到分发的一站式授课模块。在学生端创新性地集成了“终端操作+报告编写”同屏双栏的工作台(内嵌了 Markdown 富文本编辑器与 xterm.js 智能终端),并实现了作业的自动定时保存与精准防逾期校验,打通了“布置-实验-提交-批改”的线上闭环。

——基于 RAG 技术的大模型智能辅导集成:引入大语言模型(LLM)赋能实验答疑环节,利用检索增强生成(RAG)技术,将上传的实验指导书解析为向量化知识库(ChromaDB)。在学生提问时,系统动态检索相关上下文并注入 System Prompt 中,有效抑制了模型幻觉,使其能够基于本地知识手册给出准确且具有针对性的操作指引。

7.2 不足之处

由于受限开发周期与硬件资源,本系统在全面性与容错性方面仍有进一步完善的空间,主要表现在以下两点:

——集群存储管理能力局限:系统目前对存储资源的管控尚停留在本地文件系统与基础网络维度。对于支撑高可靠性场景的 Ceph 分布式存储架构,由于其涉及的调度逻辑较为繁琐,本平台暂未实现其在 Web 端的可视化纳管,在参数精细化配置与深度状态监测方面仍存在功能缺口。高级集群存储组件的纳管深度不足:目前系统主要纳管了 PVE 集群的基础网络配置与本地文件系统。对于企业级高可用场景中高度依赖的分布式存储架构(如 Ceph),因其配置逻辑的复杂性,目前尚未在系统前后端中实现深度的视图化封装与参数调优功能。

——目前系统监控体系尚处于以仪表盘(Dashboard)数据可视化为主的被动阶段,缺乏主动干预手段。当面临虚拟机资源耗尽、容器脱机或非预期宕机等关键故障时,由于尚未集成基于钉钉、企业微信等 Webhook 协议的即时告警机制,导致系统在突发状况下存在预警真空。此外,由于自动化巡检模块的缺失,平台难以在静默状态下实现对集群运行风险的实时捕捉与精准推送。缺乏多渠道主动告警与自动化巡检机制:当前系统的异常监控主要依赖于用户在 Dashboard 中的被动数据查看与图表渲染。对于虚拟机集群资源枯竭、容器失联或异常关机突发状况,尚未接入钉钉、企业微信等第三方 Webhook 推送渠道,缺少一套具有即时性和静默值守能力的自动化预警模块。

7.3 未来展望

——结合当前云原生技术的快速演进以及智能运维(AIOps)的发展趋势,本平台在未来的系统迭代中计划向以下三个方向进行技术探索:

——异构云原生底座的融合纳管(Kubernetes 扩展):在现有管理 QEMU 虚拟机和 LXC 容器的基础上,未来计划通过深度适配打通 Kubernetes 接口。使系统能够下发和编排隔离度更高、启动更轻量的 Docker 容器或 Pod,为开发者提供双轨制(重核虚拟机与轻量级容器池)的并发实验算力选择模式。

——集成底层全链路数据灾备体系(PBS 联动):为进一步确保教学实验平台的数据安全,未来可尝试与 Proxmox Backup Server(PBS)进行系统级接口集成。实现对全量实验模板、重要节点系统的计划性冷热备份和增量数据快照同步,建立具备抗毁异地容灾级别的教学平台数据保留策略。

——探索 Agentic AI 面向运维操作的自主执行能力:在具备上下文 RAG 问答的基础上,进一步提升 AI 管家的行为边界,使其向具备底层执行操作能力的智能体(Agent)进化。通过对安全操作指令集(如网络重置、基础服务重启)进行严格的 Token 授权约束,允许语言模型解析学生意图后自主调用 PVE 调整命令,完成基础报错抢修,真正实现具备一定自愈拯救能力的教学陪伴系统。

参考文献 Comment by mecio: 文献要补足20篇。

[1] Abdurrahman, Soni, Afdhil Hafid. OPTIMALISASI SUMBER DAYA KOMPUTER DENGAN VIRTUALISASI SERVER MENGGUNAKAN PROXMOX VE [J]. JURNAL FASILKOM, 2019, 8 (2) : 369-376.

[2] 李朝兵, 张涛涛, 都升升. Proxmox虚拟化平台简介 [J]. 研究·技术与应用, 2018 (10) : 50-53.

[3] Simon M, Huraj L. VirtualBox and Proxmox VE in Network Management: A User-Centered Comparison for University Environments [C]. In: Silhavy R, Silhavy P (eds) CSOC 2023. LNNS, vol 723. Springer, Cham, 2023: 486-495.

[4] 游钊栗. Windows平台下基于Proxmox虚拟化环境的云桌面系统设计与应用 [D]. 硕士学位论文, 电子信息(控制工程), 信息科学与工程学院, 2023.

[5] 陈书明, 王晶, 余少勇, 等. 高校计算机实验教学系统云管理平台设计与实现——以龙岩学院为例 [J]. 龙岩学院学

报, 2025, 43 (2) : 91-101.

[6] 王磊. 基于Proxmox VE超融合虚仿教学实训平台的设计与实现 [J] . 科学技术创新, 2025 (10) : 113-116.

[7] 张振锋. 基于Proxmox超融合虚拟化平台的设计与实现——以网络靶场竞技系统应用为例 [J] . 现代信息技术, 2020, 4 (20) : 131-134.

[8] 梁晟, 方凯明. 基于Proxmox的云平台设计与实现 [J] . 贵阳学院学报 (自然科学版), 2017, 12 (4) : 23-26.

[9] 高安邦. 基于PVE的中职网络攻防技术课程云实训平台的设计与实现 [J] . 广东教育·职教, 2021 (3) : 70-72.

[10] 陈顺荣, 周成江, 王伟. 基于虚拟化技术多功能云平台的设计研究实验 [J] . 软件, 2023, 44 (6) : 181-186.

[11] 王文选. 小微分支机构采用PVE私有云开源架构的技术方案 [J] . 金融科技时代, 2021 (9) : 58-61.

[12] Proxmox VE官方文档. Proxmox Server Solutions GmbH. [在线] . 可

用: https://pve.proxmox.com/wiki/Main_Page

[13] 北京庭宇科技有限公司.Proxmox VE赋能的弹性私有云资源调度系统:CN119473515A [P/OL] .2025-02-18 [2026-04-13] .<https://www.cqvip.com/doc/patent/00029HCIMNDG0J9W6JCG8J4P9Y>.

[14] Ariyanto, Yuri.SINGLE SERVER-SIDE AND MULTIPLE VIRTUAL SERVER-SIDE ARCHITECTURES: PERFORMANCE ANALYSIS ON PROXMOX VE FOR E-LEARNING SYSTEMS [J] .ITEGAM-JETIA,2023,9 (44) :25-34.DOI:10.5935/jetia.v9i44.903.

[15] Simon, Marek,Huraj, Ladislav.VirtualBox and Proxmox VE in Network Management: A User-Centered Comparison for University Environments [C] .Lecture Notes in Networks and Systems.Niger,2023: 486-495.DOI:10.1007/978-3-031-35317-8_44.

[16] Kovari, A.,Dukan, P..KVM & OpenVZ virtualization based IaaS open source cloud virtualization platforms: OpenNode, Proxmox VE [C] .2012 IEEE 10th Jubilee International Symposium on Intelligent Systems and Informatics, SISY 2012.Canada,2012:335-339.DOI:10.1109/SISY.2012.6339540.

[17] Oleksiuk, Vasyl P.,Spirin, Oleg M.,Holovnia, Olena S., et al.Evaluating the effectiveness of a cloud-based laboratory for teaching Linux operating systems to Computer Science students [C] .CEUR Workshop Proceedings.Republic of Korea,2024:111-126.

[18] Baun, Christian,Kappes, Martin,Cocos, Henry-Norbert, et al.DESIGN AND IMPLEMENTATION OF A VIRTUAL LAB PLATFORM FOR ACADEMIC TEACHING: EXPERIENCE IN THE DEVELOPMENT AND USE OF A NEW PLATFORM FOR THE DIGITIZATION OF LABORATORIES FOR COMPUTER SCIENCE CLASSES [C] .Proceedings of the International Conferences on Applied Computing and WWW/Internet 2024.Greece,2024:3-12.

[19] Algarni, Sultan Abdullah,Ikbal, Mohammad Rafi,Alroobaea, Roobaea, et al.Performance evaluation of Xen, KVM, and proxmox hypervisors [J] .International Journal of Open Source Software and Processes,2018,9 (2) :39-54.DOI:10.4018/IJOSSP.2018040103.

[20] Chang, Bao Rong,Tsai, Hsiu-Fen,Chen, Chi-Ming, et al.Assessment of hypervisor and shared storage for cloud computing server [C] .Proceedings - 3rd International Conference on Innovations in Bio-Inspired Computing and Applications, IBICA 2012.Italy,2012:67-72.DOI:10.1109/IBICA.2012.68.

致谢

寒来暑往,四载寒窗生涯已近尾声。值此学位论文定稿与系统研发告一段落之际,回望求学之路,内心感慨万千,诸多感激之情难以言表光阴荏苒,大学四年的时光转瞬即逝。在完成这篇毕业论文和相关系统的开发之际,我心中充满了感激之情。

首先,我要向我的指导老师表达最诚挚的谢意。在系统的技术选型、架构设计以及论文的选题与撰写过程中,导师给予了我悉心的指导和无私的帮助。老师渊博的专业知识、严谨的治学态度和对开源技术前沿的敏锐洞察力,使我受益匪浅,同时也让本系统能够跳脱死板的监控概念范畴,在教务实验平台这一垂直的业务场景中找到了真正的应用落地生机与结合创新点。

其次,我要感谢大学期间各位授课教师的辛勤教导。正是他们在计算机科学、网络工程及软件开发等基础与进阶专业中所传授的基础理论与实践经验,为我能够具备全栈视野,且独立完成本次本系统从前端 UI 操作展示到后端代理开发,并能顺利处理复杂的底层服务器集群长并发等各类死锁技术难题,打下了无可替代的坚实专业基石底座基础。

此外,还要重点极其感谢当今年代充满无数活力共享与无私精神的开源世界社区及所有参与构筑这块大厦的无名英雄们(尤其是如 Proxmox VE 生态社群, 前端 Vue.js / Vben 生态社区, Django 及各类繁多的 AI 处理开源包贡献者成员等)。正所谓是站在巨人的肩膀上方能更快速地去触碰星空,我才得以在这个繁杂的信息爆炸与技术红海时代,如用搭乐高拼图的方式一砖一瓦快速高阶验证和把这庞大的具有各种底层超能力的混合虚拟化集群云管实验辅助平台成功编织搭建拼凑落地。社区论坛及开发者群内互不曾谋面者的无极互助探讨精神以及官方上详实中肯的海量指引文章资料,更是陪伴着我在这漫长探索期并一盏又一盏灯般引导我跨越无数个报错堆栈调试与代码死锁悬崖。

最后,深深地感谢我的父母以及始终关心我、默默帮持过我的好朋友、室友还有同学们。正是因为你们长期的包容、深厚的理解、在迷茫疲惫时候所给予温暖的无声鼓励陪伴慰藉,给予了我不断地去去钻去死磕难点地无穷极的力量深水源泉;让我在以往曾无数个被毫无头绪的框架 Bug 折腾到难以安眠且充满无数未知的深夜赶路求学日子里可以保持一颗从容向前的坚韧初心去迈步前行不馁败走。

原创性声明

本人郑重声明:所提交的毕业论文(设计),是本人在导师的指导下进行的研究工作及取得的成果。除文中已经注明引用的内容外,本论文(设计)不含其他个人或机构已经发表或撰写过的研究成果,也没有剽窃、抄袭等违反学术道德规范的侵权行为。对本论文(设计)的研究做出重要贡献的个人和集体,均已在文中以明确方式标明。本人愿意承担由本声明而引起的法律责任。

学生签名: 日期: 年 月 日